Contents

List of Tables **vi**

List of Figures **viii**

Chapter 1: Introduction **1**
1.1 Project Details 1
1.2 Purpose 1
1.3 Scope 1
1.4 Objectives 2
1.5 Technology Stack Used 2
1.6 Problem Context and Motivation 3
1.7 Literature Review 3
1.7.1 Digital Governance and Complaint Systems 3
1.7.2 Vision-Language Models in Public Workflows 4
1.7.3 Local Inference and Privacy Considerations 4
1.7.4 Hybrid Deterministic + AI Pipelines 4
1.7.5 Governance, Explainability, and Auditability 4
1.7.6 System Comparison 5
1.8 Chapter Summary 5

Chapter 2: Project Management **6**
2.1 Feasibility Study 6
2.1.1 Technical Feasibility 6
2.1.2 Time Schedule Feasibility 7
2.1.3 Operational Feasibility 7
2.1.4 Implementation Feasibility 7
2.2 Project Planning 7
2.2.1 Development Approach and Justification 7
2.2.2 Milestones and Deliverables 8
2.2.3 Project Timeline (Gantt Chart) 9
2.2.4 Group Dependencies 10
2.3 Risk Management 10
2.4 Management Outcomes 11
2.5 Management Retrospective 11
2.6 Chapter Summary 12

Chapter 3: System Requirement Study **13**

3.1 Existing System Overview 13
3.2 Limitations of the Existing System 13
3.3 User Characteristics 13
3.4 Functional Requirements 14
3.5 Non-Functional Requirements 15
3.6 Hardware and Software Requirements 16
3.6.1 Hardware Requirements 16
3.6.2 Software Requirements 17
3.7 Constraints 17
3.7.1 UI Constraints 18
3.7.2 Communication Interface 18
3.7.3 Hardware Interface 18
3.7.4 Criticality of Application 18
3.7.5 Safety and Security Considerations 18
3.8 Assumptions and Dependencies 19
3.9 Detailed Use-Case Narratives 19
3.10 Requirement Priority Classification 20
3.11 Chapter Summary 21

Chapter 4: Proposed System Requirements 22
4.1 Overview of Proposed System 22
4.2 Module Descriptions 22
4.2.1 Authentication and User Management Module 22
4.2.2 Analyze and Complaint Intake Module 23
4.2.3 Complaint Lifecycle Module 23
4.2.4 Assignment and Worker Operations Module 23
4.2.5 Notification and Communication Module 23
4.2.6 Triage and Governance Module 23
4.2.7 Analytics and Public Transparency Module 23
4.3 System Features 23
4.4 Advantages of Proposed System 24
4.4.1 Citizen-Centric Advantages 25
4.4.2 Administrative and Departmental Advantages 25
4.4.3 Institutional Advantages 25
4.5 Requirement Realization Matrix 25
4.6 Proposed Operational Workflow 26
4.7 Expected Impact 26
4.8 Module Interaction Contracts 27
4.9 Requirement Realization Detail by Persona 27
4.10 Proposed Data and Control Flow Narrative 28
4.11 Chapter Summary 28

Chapter 5: System Design 29
5.1 System Architecture Design 29

5.1.1 Architectural Style 29
5.1.2 Key Architectural Decisions 30
5.1.3 AI Classification Pipeline 30
5.2 UML Diagrams 31
5.2.1 E-R Diagram 31
5.2.2 Use Case Diagram 32
5.2.3 Class Diagram 32
5.2.4 Sequence Diagram 33
5.2.5 Activity Diagram 33
5.2.6 DFD Diagram 34
5.2.7 Deployment Diagram 34
5.3 Database Design 35
5.3.1 Table Design and Relationships 35
5.3.2 Normalization 35
5.3.3 Indexing Strategy 35
5.4 GUI Design 36
5.4.1 Design Principles 36
5.4.2 Interface Areas 37
5.4.3 Wireframe Reference 37
5.5 Screenshots 38
5.6 Chapter Summary 41

Chapter 6: Implementation Planning **42**
6.1 Implementation Environment 42
6.1.1 Local Development Environment 42
6.1.2 Production-Style Container Environment 42
6.2 Tools & Technologies Used 43
6.3 Key Implementation Patterns 44
6.3.1 Upload Validation Chain 44
6.3.2 Classifier Confidence Thresholding 44
6.3.3 Frontend Route Guard 45
6.4 Coding Standards Followed 45
6.4.1 Backend Standards 45
6.4.2 Frontend Standards 45
6.4.3 Configuration Standards 45
6.5 Implementation Work Packages 46
6.5.1 Work Package 1: Core Backend and Data Layer 46
6.5.2 Work Package 2: AI Pipeline Integration 46
6.5.3 Work Package 3: Frontend Role Flows 46
6.5.4 Work Package 4: Operational Features 46
6.5.5 Work Package 5: Security and Reliability Hardening 46
6.5.6 Work Package 6: Deployment and Handover Readiness 46
6.6 Implementation Sequence and Dependencies 47
6.7 Configuration and Secret Planning 47

6.8 Quality Gates in Implementation Plan 48
6.9 Resource and Effort Planning 48
6.10 Detailed Module Implementation Notes 48
6.10.1 Backend Router Implementation Notes 48
6.10.2 Service-Layer Planning Notes 49
6.11 Deployment Runbook and Rollout Plan 50
6.11.1 Rollout Stages 50
6.11.2 Rollback Planning 51
6.12 Operational Monitoring Plan 51
6.13 Change Management and Release Strategy 52
6.13.1 Release Checklist 52
6.13.2 Implementation Learnings 52
6.14 Chapter Summary 53

Chapter 7: Testing **54**
7.1 Testing Plan 54
7.1.1 Testing Objectives 54
7.1.2 Test Environment 54
7.1.3 Coverage Matrix 55
7.2 Types of Testing 56
7.2.1 Unit Testing 56
7.2.2 Integration Testing 56
7.2.3 System Testing 57
7.2.4 User Acceptance Testing (UAT) 57
7.2.5 Performance and Load SLA Metrics 57
7.3 Testing Techniques 58
7.3.1 Black-Box Behavioral Testing 58
7.3.2 Negative Testing 58
7.3.3 Resilience and Fault Injection 58
7.3.4 Performance and Load Testing 59
7.3.5 System Evaluation Metrics 59
7.4 Testing Outcomes 60
7.5 Residual Risks and Test Gaps 60
7.6 Detailed UAT Case Matrix 61
7.7 Load Benchmark Profiles and Acceptance Rubric 63
7.8 Security Regression Matrix 63
7.9 Chapter Summary 64

Chapter 8: Limitations and Future Scope **65**
8.1 Current Limitations 65
8.1.1 Model and AI Limitations 65
8.1.2 Infrastructure Limitations 65
8.1.3 Workflow and Data Limitations 65
8.1.4 Security and Governance Limitations 66

8.1.5 Real-World Deployment Challenges 66
8.2 Future Scope 66
8.2.1 AI and Intelligence Enhancements 66
8.2.2 Operational Workflow Enhancements 67
8.2.3 Platform and Deployment Enhancements 67
8.2.4 Security and Compliance Enhancements 67
8.3 Roadmap Perspective 68
8.4 Risk-Prioritized Improvement Backlog 68
8.5 Adoption Considerations for Municipal Deployment 70
8.5.1 Institutional Readiness Factors 70
8.5.2 Change Management Considerations 70
8.5.3 Data Governance Considerations 70
8.6 Research Continuation Opportunities 70
8.7 Chapter Summary 70

Chapter 9: Conclusion and References **71**
9.1 Conclusion 71
9.2 References 71

List of Tables

1.1 Comparison of Grievance System Architectures 5

2.1 Milestone Summary and Deliverables 8
2.2 Role Responsibility Matrix (Project Execution) 10
2.3 Risk Register and Mitigation Plan 11

3.1 Functional Requirement Specification 14
3.2 Non-Functional Requirement Specification 16
3.3 Hardware Requirement Baseline 17
3.4 Software Environment Requirements 17
3.5 Assumptions and External Dependencies 19
3.6 Narrative Use-Case Requirements 19
3.7 MoSCoW-Style Requirement Priority Mapping 21

4.1 Proposed Feature Set and Operational Benefit 23
4.2 Traceability from Requirements to Proposed Modules 25
4.3 Inter-Module Contract Summary 27

5.1 Major Design Decisions and Rationale 30
5.2 Indexing Strategy by Collection 36
5.3 GUI Areas and Purpose 37

6.1 Implementation Toolchain and Use Cases 43
6.2 Implementation Sequence with Dependency Notes 47
6.3 Implementation Quality Gates 48
6.4 Router-Level Implementation Planning Notes 49
6.5 Service-Level Implementation Planning 50
6.6 Deployment Rollout Stages 51
6.7 Operational Monitoring Matrix 51

7.1 Testing Environment Baseline 55
7.2 Testing Coverage Matrix by Suite 55
7.3 Representative Unit Test Cases 56
7.4 System Test Scenarios 57
7.5 System SLA and Performance Metrics 58
7.6 System Evaluation Metrics 59
7.7 Outcome Summary by Validation Area 60
7.8 Persona-Driven UAT Matrix 61
7.9 Load Profile and Acceptance Guidance 63

7.10 Security Regression Verification Matrix 63

8.1 Indicative Post-Submission Roadmap 68
8.2 Future Backlog Prioritization Matrix 68

List of Figures

2.1 Project Gantt Chart — Week-by-Week Delivery Schedule 9

4.1 Proposed End-to-End Workflow 26

5.1 Overall System Context Diagram 29
5.2 Component Architecture Diagram 30
5.3 E-R Diagram 31
5.4 Use Case Diagram 32
5.5 Class Diagram 32
5.6 Sequence Diagram 33
5.7 Activity Diagram 33
5.8 Data Flow Diagram 34
5.9 Deployment Diagram 34
5.10 GUI Wireframe Reference from Project Documentation 38

7.1 Response Time vs Load 59
7.2 Classification Accuracy vs Test Samples 60

List of Screenshots

5.1 Home Page Hero 38
5.2 AI Analysis Result 39
5.3 Citizen Tracking Dashboard 39
5.4 Worker Assignment Queue 40
5.5 Admin Dashboard 40
5.6 Admin Triage Queue 41

Chapter 1

Introduction

Jan-Sunwai AI is an end-to-end civic grievance redressal platform built to reduce the operational gap between citizen complaint submission and departmental action. The system transforms an image upload into a structured, auditable complaint workflow, then routes the case to accountable municipal actors. Instead of requiring citizens to understand bureaucratic departments in advance, the platform performs AI-assisted classification and drafting while preserving a human-governed correction path through triage and administrative controls [11, 12, 13, 17, 16].

The project was developed as a local-first system to satisfy practical constraints in municipal settings where cloud deployment may be restricted by policy, infrastructure, or privacy concerns. The architecture integrates a React frontend, FastAPI backend, MongoDB persistence, and locally hosted Ollama models in a hybrid inference pipeline [11, 12, 13, 17, 16]. This deployment posture allows the platform to run in constrained institutional environments while maintaining modularity for future expansion.

1.1 Project Details

Jan-Sunwai AI is a civic grievance management platform targeting **NDMC-style municipal governance operations**, developed over the period **28 January to 27 May 2026**. Its application domain is civic grievance management and municipal operations, with primary users spanning citizens, workers, department heads, and administrators. The system is deployed as a **local-first, containerised stack** designed to operate without dependency on managed cloud services.

1.2 Purpose

Traditional complaint systems rely heavily on manual typing, manual category selection, and office-level routing knowledge that many citizens do not have. This results in poor-quality submissions, delayed routing, and low trust in resolution timelines. The purpose of Jan-Sunwai AI is to build a practical, auditable, and secure system where:

- Citizens can submit evidence-driven complaints quickly using photos.
- The system auto-detects likely department categories and drafts complaint text.
- Authorities can monitor, assign, escalate, and resolve complaints with full history.
- Governance stakeholders can access analytics, public transparency views, and exports.

The objective is not to replace human decision-making but to reduce repetitive effort and routing ambiguity while strengthening accountability.

1.3 Scope

The implemented scope includes intake, classification, routing metadata, assignment, notifications, escalation support, and governance analytics. Specifically, the platform covers:

- User management with role-aware access control.
- AI analysis and complaint drafting from uploaded images.
- Complaint lifecycle actions: create, track, update, transfer, escalate, and close.
- Department-specific work queues and worker assignment controls.
- Triage process for low-confidence or ambiguous classifications.
- Notification chain and password reset mechanisms.
- Public-facing anonymized complaint listing and operational analytics.

Out-of-scope items for the current release include multilingual voice intake, direct integration with external municipal ERP systems, and predictive resource planning beyond dashboard analytics.

1.4 Objectives

The platform was built to satisfy seven core objectives:

1. **Reduce submission friction** — Simplified upload-analyze-submit flow with optional draft editing.
2. **Improve complaint routing quality** — Canonical department taxonomy and authority mapping.
3. **Ensure operational traceability** — Status history, notes, comments, and an audit-ready timeline.
4. **Support workload distribution** — Auto-assignment by department, service area, and load balancing.
5. **Strengthen security posture** — Cookie-session auth, input sanitization, upload validation, and CORS/security headers.
6. **Improve governance transparency** — Analytics dashboard, heatmap, and public complaint feed.
7. **Enable production transition** — Docker-based deployment profile, environment controls, and runbooks.

1.5 Technology Stack Used

The system is built on a five-layer stack. The **frontend** uses React and Vite for role-based dashboards and authentication-aware routing. The **backend API** is built with FastAPI and Python, handling REST endpoints, validation, and middleware. **MongoDB** provides document storage for users, complaints, notifications, and reset tokens. **Ollama** models serve as the local AI runtime for vision extraction, reasoning, and draft generation. **Docker Compose** orchestrates the full stack for reproducible dev and production deployment. Testing is handled by pytest, httpx, and locust, covering unit, integration, security, resilience, and load scenarios.

1.6 Problem Context and Motivation

Civic systems face a recurring quality bottleneck at intake. According to India's CPGRAMS annual report, over 19 lakh (1.9 million) grievances were registered in 2022–2023 alone, with a significant proportion requiring re-assignment after initial routing errors [1, 2]. At the municipal level, studies of urban local body portals show that miscategorized complaints take an average of 3–5 additional working days to resolve compared to correctly filed ones, as each wrong-department transfer adds a full review cycle. In high-volume departments such as Civil Works or Vector-Borne Disease Control, this translation overhead directly delays field action.

Jan-Sunwai AI addresses this bottleneck by introducing structured automation at the earliest stage while preserving administrative authority. By combining vision-language classification with a deterministic routing engine, the platform reduces the cognitive burden on citizens (who typically do not know which department owns a given infrastructure issue) while giving administrators a governed, auditable triage path for edge cases. This balance between automation and human control is the central design philosophy of the system.

1.7 Literature Review

The literature review for this project focuses on civic-tech complaint systems, AI-assisted classification workflows, and governance requirements such as traceability and explainability. Rather than treating model accuracy as the only success metric, the review emphasizes operational outcomes such as routing correctness, audit readiness, and response-time reliability in public-sector environments. The selected architectural direction in Jan-Sunwai AI was derived from this practical lens [1, 2, 19, 9].

1.7.1 Digital Governance and Complaint Systems

Digital grievance platforms are a recognized mechanism for increasing civic participation and service responsiveness. At the national level, frameworks like India's Centralised Public Grievance Redress and Monitoring System (CPGRAMS) have set precedents for digital complaint routing across ministerial departments. Similarly, under the Smart Cities Mission, municipal bodies and the National Informatics Centre (NIC) have deployed local civic portals aimed at bringing e-governance directly to citizens. However, despite these advancements, many of these systems remain structurally form-driven and text-heavy. They require citizens to possess a high degree of procedural awareness to correctly select the responsible department from complex drop-down menus, creating a significant barrier to entry for users with low digital literacy.

Literature and practitioner reports consistently emphasize that the quality of the initial citizen complaint is directly linked to downstream processing efficiency [1, 2]. When a grievance is miscategorized at intake, it enters a slow bureaucratic loop of inter-departmental transfers. Jan-Sunwai AI seeks to address this specific gap in the Indian e-governance landscape by replacing text-heavy categorization forms with an intuitive, image-first interface that shifts the cognitive load of classification from the citizen to the machine.

1.7.2 Vision-Language Models in Public Workflows

While foundational vision-language models (VLMs) like CLIP [3] and Llama 3.2 Vision [4] have demonstrated remarkable zero-shot reasoning capabilities on generalized internet data, applying them directly to public administrative workflows introduces novel challenges. A VLM might correctly identify a ”broken pipe leaking water onto a road,” but lack the institutional context to determine whether the issue belongs to the Jal Board (Water Authority) or the Public Works Department (Roads). Consequently, purely generative approaches often hallucinate jurisdictional boundaries. Prior practical studies in neuro-symbolic systems suggest that the most reliable approach is to use the VLM strictly for context extraction, while relying on deterministic, rule-based control layers to execute the actual governance routing [5].

1.7.3 Local Inference and Privacy Considerations

Public institutions frequently operate under constraints where sensitive citizen data should remain within controlled infrastructure boundaries. Local inference minimizes external data transfer and provides better control over model lifecycle, though it introduces operational concerns around VRAM budgeting, process health, and failover behavior. This project adopts local inference as a policy-aware technical choice rather than only a performance decision.

1.7.4 Hybrid Deterministic + AI Pipelines

To mitigate the unreliability of end-to-end deep learning models in high-stakes environments, hybrid pipelines integrate neural networks with symbolic reasoning architectures [5]. In this staged approach, a vision model is responsible only for perceptual tasks—such as detecting debris or structural damage—while a traditional rules engine handles the logical task of mapping those detections to specific departmental SLAs. This separation of concerns significantly reduces the hallucination risk inherent to large language models. Jan-Sunwai AI adopts this hybrid pattern to guarantee that even if the AI misinterprets an image, the resulting classification remains within a bounded set of valid municipal categories, ensuring the system behaves predictably under audit.

1.7.5 Governance, Explainability, and Auditability

Unlike commercial software, civic applications operate under strict mandates for public accountability and transparency. When an automated system denies or reroutes a citizen’s service request, the decision cannot be a black box; explainability is a legal and ethical requirement. Research into Human-in-the-Loop (HITL) machine learning strongly advocates for systems where AI acts as a recommender rather than an autonomous agent, especially in domains affecting public welfare [6]. Consequently, Jan-Sunwai AI is designed with exhaustive decision provenance. Every state transition logs the actor, the timestamp, and—in the case of auto-routing—the exact confidence threshold that triggered the action. By forcing low-confidence AI decisions into a manual triage queue, the platform embodies HITL principles, ensuring that human administrators remain the ultimate arbiters of complex civic disputes [1, 9].

1.7.6 System Comparison

To validate the necessity of the proposed architecture and transition from descriptive to analytical literature, we compare traditional grievance portals with the Jan-Sunwai AI hybrid system:

Table 1.1: Comparison of Grievance System Architectures

System Attribute	Traditional Portal	Pure Cloud AI System	Proposed Hybrid System
Input Type	Manual Text/Forms	Unstructured Text/Image	Image + Text (AI assisted)
Classification	Manual Selection	Non-deterministic LLM	Hybrid (Rules + Local VLM)
Routing Accuracy	Low (Misrouting common)	Variable (Hallucination risk)	High (Governed rules)
Infrastructure	Low requirement	High (Cloud APIs needed)	Moderate (Local GPU inference)
Primary Limitation	High user friction	Privacy concerns, unpredictable	Requires specialized local hardware

The project design aligns with development principles and platform guidance from FastAPI, React, MongoDB, and OWASP references listed in Chapter 9 [11, 12, 13, 7, 8].

1.8 Chapter Summary

This chapter defined the problem setting, project intent, scope boundaries, and technology foundation. It also established the literature-aligned rationale for adopting a hybrid AI plus deterministic architecture in a governance-sensitive municipal context. The next chapter details project planning, feasibility analysis, schedules, and delivery governance.

Chapter 2

Project Management

Project management for Jan-Sunwai AI followed an iterative delivery approach with explicit weekly milestones, acceptance checkpoints, and operational hardening gates. The planning strategy balanced feature development with risk reduction, especially in AI reliability, security posture, and deployment reproducibility.

2.1 Feasibility Study

The feasibility study was carried out before deep implementation to ensure that the project could be delivered within academic timelines while still meeting production-oriented quality goals. Evaluation covered technical viability, institutional adoption realities, timeline sustainability, and deployment reproducibility. The findings confirmed that a local-first architecture was feasible, provided that model orchestration, security controls, and environment standardization were handled as first-class concerns.

2.1.1 Technical Feasibility

The selected stack (React, FastAPI, MongoDB, Ollama) was technically feasible for local-first civic deployment due to mature ecosystem support, strong documentation, and compatibility with available development infrastructure [11, 12, 13, 17, 16]. Key feasibility observations were:

- FastAPI provided rapid API development, typed schemas, strict `/api/v1` namespace versioning, and OpenAPI generation.
- MongoDB supported flexible document structures suited to complaint histories, comments, and embedded metadata.
- React (v18 with React Router v7) enabled role-specific UI composition with route guards and dashboard specialization.
- Vite bundled the application with extreme speed, while Tailwind CSS v4 provided aggressive, utility-first UI styling.
- Ollama provided local model execution with an in-memory background processing queue.
- slowapi was integrated natively into the FastAPI lifecycle to handle Rate Limiting effectively.
- Docker Compose enabled reproducible environment orchestration across development and production-like setups.

A major early technical risk was model reliability under constrained VRAM. This was mitigated by introducing a staged inference pipeline and avoiding simultaneous heavy model loading.

2.1.2 Time Schedule Feasibility

The schedule was structured over 18 weeks, progressing from foundation to hardening and final submission. Work cadence used Monday–Saturday windows with planned Sunday off-days. Alternate Saturday breaks were also reserved. This realistic scheduling improved sustainability while preserving delivery continuity.

2.1.3 Operational Feasibility

Operational feasibility was validated by mapping municipal personas into role-based workflows:

- Citizen: complaint intake and tracking.
- Worker: assignment execution and completion updates.
- Department Head: queue supervision and status transitions.
- Admin: approvals, triage governance, analytics, and exports.

The resulting workflow matched expected governance practices where supervisory roles require both visibility and controlled override capabilities.

2.1.4 Implementation Feasibility

Implementation was feasible under both manual and scripted setup modes. Automation scripts were created for Windows and Linux, and production deployment was validated through containerized orchestration [16]. Environment configuration was standardized via a single backend environment file to reduce drift and support repeatable handover.

2.2 Project Planning

Project planning translated broad objectives into actionable delivery slices with explicit milestones, quality gates, and integration checkpoints. Planning was intentionally cross-functional, so backend APIs, frontend role flows, AI services, and deployment scripts evolved in synchronized increments rather than in isolated tracks. This structure reduced integration shocks and enabled predictable weekly review cycles.

2.2.1 Development Approach and Justification

The project used an incremental and verification-driven approach with the following cycle:

1. Build baseline functionality for a module.
2. Validate through unit and integration checks.
3. Expose role workflow in UI and test end-to-end behavior.
4. Add security/performance hardening after functional stabilization.
5. Document operational procedures and release notes.

This approach was chosen over a strict waterfall model because AI behavior and UX patterns required iterative calibration [17, 11, 12].

2.2.2 Milestones and Deliverables

The milestone table below maps each delivery phase to its planned completion gate and primary acceptance criterion.

Table 2.1: Milestone Summary and Deliverables

Window	Theme	Key Deliverables	Status
Weeks 1–2	Foundation	Repo setup, environment bootstrap, backend skeleton, DB connectivity, auth schema, complaint schema, initial CRUD and upload service	Completed
Weeks 3–5	AI Pipeline	Model research, classifier iterations, geotagging, reverse geocoding, generation service, analyze endpoint integration	Completed
Week 6	Frontend Base	Routing, upload flow, analyze hook, result view, integration with backend auth and analyze pipeline	Completed
Week 7	Flow Closure	Map marker improvements, local draft persistence, error boundary, 404 route, full persona walkthrough	Completed
Week 8	Security Layer	Upload hardening, rate limiting, sanitization, CORS lockdown, security headers, index hardening	Completed
Week 9	NDMC Feature Pack	Worker panel, assignment engine, audit trail, analytics/heatmap, SLA badges, bulk actions/exports	Completed
Week 10	Notification and Communication	Navbar badge, status-triggered notifications, email stub, forgot/reset password, profile editing, E2E notification chain test	Completed
Week 11	Performance and Reliability	Image compression, lazy loading, graceful 503 degrade, profiling, resilience tests	Completed
Week 12	Production Hardening	Docker production files, compose profile, env plan, API versioning, backup strategy, UI audit	Completed
Weeks 13–14	Testing and Pre-Handover	Unit/integration/security/load tests, mobile fixes, handover guide, API reference, repository cleanup, CI workflows	Completed

Window	Theme	Key Deliverables	Status
Weeks 15–16	UAT and Staging Validation	Clean deploy simulation, production verification, UAT persona runs, feedback loop and polish pass	Completed
Weeks 17–18	Reporting and Submission	Technical report artifacts, screenshots, presentation, final submission hygiene and release tagging	Completed

2.2.3 Project Timeline (Gantt Chart)

The Gantt chart below provides a visual representation of the week-by-week delivery schedule, illustrating how development streams overlapped and how hardening, testing, and documentation phases were sequenced relative to feature delivery.

[Gantt Chart — Insert Image Here]

Save your image as: `docs/images/screenshots/gantt_chart.png`

Then replace this entire `\fbox{...}` block with:

```
\includegraphics[width=0.92\textwidth]
{../../images/screenshots/gantt_chart.png}
```

Figure 2.1: Project Gantt Chart — Week-by-Week Delivery Schedule

The structured breakdown below maps the project lifecycle into actionable, tracked deliverables.

Role Area	Responsibilities
Backend Engineering	API design, schema validation, business logic, role authorization, lifecycle transitions, notification hooks
Frontend Engineering	Route architecture, role dashboards, analyze/result flow, user session handling, responsive behavior
AI and Classification	Vision model integration, rule engine tuning, reasoning fallback, draft generation quality checks
Data and Persistence	MongoDB collection design, indexes, query access paths, reset token TTL and audit metadata
Security and Reliability	Input sanitization, upload hardening, CORS and headers, degrade behavior, readiness/liveness semantics
Testing and QA	Unit and integration tests, resilience and security validation, load profiling, UAT scripts
DevOps and Deployment	Compose consolidation, environment normalization, production verification scripts, backup runbooks
Documentation and Handover	API reference, deployment guides, user manual, architecture and report compilation

Table 2.2: Role Responsibility Matrix (Project Execution)

2.2.4 Group Dependencies

The project includes cross-cutting dependencies that were managed explicitly:

- Backend depends on MongoDB availability and correctly configured environment variables.
- Analyze flow depends on file storage service and Ollama runtime health.
- Frontend route behavior depends on cookie-session state and role claims.
- Assignment quality depends on worker profile completeness (department, approval, service area, status).
- Production readiness depends on compose health checks, proxy alignment, and secure secret configuration.

Dependency risks were controlled through health endpoints, fallback behaviors, and scripted verification routines.

2.3 Risk Management

Table 2.3: Risk Register and Mitigation Plan

Risk	Likelihood	Impact	Mitigation
Model unavailability or timeout	Medium	High	Multi-tier vision fallback, queue-based generation, retryable 503 responses
Wrong department classification	Medium	High	Deterministic rule engine, triage queue, admin correction workflow
Invalid/malicious uploads	Medium	High	Extension + MIME + magic number + image decode validation, max size enforcement
Unauthorized route access	Low	High	Cookie-session auth (JWT compatibility), role-gated dependencies, integration tests for permission boundaries
Configuration drift across environments	Medium	Medium	Unified backend environment strategy and scripted deployment checks
Notification inconsistency	Low	Medium	Automated notification-chain tests and mark-all-read verification
Performance degradation under load	Medium	Medium	Profiling, timing logs, load-testing matrix, controlled fallback behavior
Documentation mismatch at handover	Medium	Medium	NDMC deployment guide, API reference sync, verification procedure chapter

2.4 Management Outcomes

The management strategy achieved three notable outcomes:

- Feature breadth was delivered without compromising security and reliability hardening.
- Delivery confidence increased through continuous testing and issue-driven iteration.
- Handover readiness improved through consolidated deployment/configuration artifacts and documentation alignment.

2.5 Management Retrospective

Three management lessons were central to delivery quality:

1. **Early role modeling reduced rework:** Defining citizen/worker/dept-head/admin path-

ways early prevented significant UI and authorization redesign later.

2. **Security and reliability should not be deferred:** Integrating these streams before final release avoided unstable late-stage merges.
3. **Artifact consolidation is a management task:** Unifying environment and compose artifacts reduced support burden and improved handover confidence.

2.6 Chapter Summary

This chapter demonstrated that the project was not only technically implementable but also deliverable within a controlled timeline and governance framework. Feasibility, dependency management, risk planning, and milestone tracking supported predictable progress toward a production-oriented civic platform.

Chapter 3

System Requirement Study

This chapter documents requirement analysis for Jan-Sunwai AI, including the baseline state of existing grievance systems, observed limitations, actor expectations, and formalized functional/non-functional requirements. The study was derived from implementation realities, municipal workflow assumptions, and validation through tests and user journeys.

3.1 Existing System Overview

Many conventional civic complaint channels depend on one or more of the following interfaces: physical counters, basic web forms, call center logging, or generic ticketing portals. While these methods can register complaints, they often suffer from weak input quality, manual routing dependency, and low traceability.

Key characteristics of existing systems observed in this problem context:

- Citizens are expected to manually choose departments, often without sufficient guidance.
- Complaint narratives are text-heavy and unstructured, leading to inconsistent detail quality.
- Photo evidence may be accepted but is rarely integrated into routing logic.
- Status updates are frequently sparse or delayed.
- Workflow metadata (who changed what and why) is often incomplete for audits.

3.2 Limitations of the Existing System

Seven recurring limitations were observed in legacy civic grievance systems: manual authority selection by citizens causing frequent misrouting; inconsistent complaint descriptions that require staff to rewrite or clarify cases before action; weak integration of visual evidence leaving photographs unused during triage; limited role-specific workflows that impose the same interface burden on very different personas; minimal lifecycle traceability making accountability reviews difficult; low transparency in queue behavior eroding citizen trust; and sparse administrative analytics that force planning decisions to rely on intuition rather than data.

3.3 User Characteristics

The system is designed for four operational personas with distinct interaction patterns and authorization boundaries.

The system serves four distinct operational personas. **Citizens** report issues and track outcomes — they upload images, review the AI-generated draft, submit their complaint, and monitor notifications and status updates. **Workers** resolve assigned field tasks by viewing their queue, managing availability, and marking completions with resolution notes. **Department Heads** manage departmental flow by filtering the department queue, updating complaint status, adding notes, and transferring or escalating cases when required. **Administrators** exercise end-

to-end governance: approving workers, triaging ambiguous AI classifications, performing bulk operations, exporting reports, and monitoring analytics.

3.4 Functional Requirements

Table 3.1: Functional Requirement Specification

FR ID	Requirement Name	Requirement Description
FR-01	User Registration	System shall allow self-registration for citizen and worker roles with validation and duplicate checks.
FR-02	User Authentication	System shall establish secure cookie-session authentication on successful login and protect authorized routes, with JWT compatibility for API clients.
FR-03	Profile Management	Authenticated users shall update selected profile fields such as full name and phone number.
FR-04	Password Recovery	System shall support forgot-password and reset-password flows with time-bound tokens.
FR-05	Image Upload Validation	Analyze endpoint shall accept valid JPG/PNG/WEBP images and reject unsupported or malformed payloads.
FR-06	AI-Assisted Analysis	System shall analyze uploaded image and return category, confidence, location metadata, and draft status.
FR-07	Draft Generation	System shall produce a complaint draft synchronously or via queued generation, with polling support.
FR-08	Complaint Creation	System shall create a complaint record from reviewed analysis payload and user-provided description edits.
FR-09	Complaint Listing	System shall provide role-filtered complaint lists with pagination and optional status filters.

FR ID	Requirement Name	Requirement Description
FR-10	Complaint Detail View	System shall provide complete complaint detail including status history and operational metadata.
FR-11	Status Update Workflow	Department head/admin shall update complaint status with note and trigger citizen notification.
FR-12	Department Transfer	Department head/admin shall transfer complaint ownership across departments when misrouted.
FR-13	Escalation	Department head/admin shall escalate complaints to parent authorities when required.
FR-14	Worker Assignment	System shall auto-assign eligible workers and support manual override by admin.
FR-15	Worker Operations	Worker shall manage availability and mark assigned complaints as done.
FR-16	Notification Center	System shall provide list, unread count, single-read, and mark-all-read operations.
FR-17	Triage Queue	Admin shall review low-confidence complaints and approve/reject/correct labels.
FR-18	Analytics	Admin/dept-head shall access overview and geospatial heatmap summaries.
FR-19	CSV Export	Admin shall export complaint data to CSV for external reporting.
FR-20	Public Transparency Feed	System shall expose anonymized complaint records without authentication.
FR-21	Health Monitoring	System shall provide live, ready, model, and GPU health endpoints.
FR-22	API Version Alias	System shall expose major endpoints under /api/v1 for integration stability.

3.5 Non-Functional Requirements

Table 3.2: Non-Functional Requirement Specification

NFR ID	Quality Attribute	Requirement Description
NFR-01	Security	Cookie-session auth (with JWT compatibility), RBAC, sanitization, secure headers, and account-safe reset flow shall be enforced.
NFR-02	Reliability	API shall degrade gracefully under model or DB faults with explicit status semantics.
NFR-03	Performance	Public and health endpoints shall respond consistently under mixed read-heavy load.
NFR-04	Usability	Role-specific UI shall reduce cognitive load and support responsive layouts for desktop/mobile usage.
NFR-05	Maintainability	Modular routers/services with documented runbooks shall support iterative enhancement.
NFR-06	Observability	Process-time headers, rotating logs, and health routes shall support debugging and operations.
NFR-07	Scalability Readiness	Queue-based generation and container deployment shall support controlled scaling.
NFR-08	Portability	Project setup shall support Windows and Linux via scripts and Docker-based workflow.
NFR-09	Compliance Readiness	Local-first inference and sanitized logs shall support stricter governance deployments.
NFR-10	Data Integrity	Validation schemas and controlled update flows shall minimize malformed or inconsistent records.

3.6 Hardware and Software Requirements

3.6.1 Hardware Requirements

The baseline parameters established below define the minimum viable constraints required to support the proposed features.

Component	Minimum	Recommended
CPU	4-core modern processor	6 to 8-core processor
System RAM	12 GB available	16 GB or higher
GPU VRAM	4 GB	6 GB or higher
Storage	10 GB free	20 GB free
Network	Stable LAN/internet for maps and package fetch	Managed institutional network

Table 3.3: Hardware Requirement Baseline

3.6.2 Software Requirements

The baseline parameters established below define the minimum viable constraints required to support the proposed features.

Software Component	Baseline	Purpose
Operating System	Windows 10+/Linux	Development and deployment host
Python Runtime	Python 3.10+	Backend and automation scripts
Node.js and npm	Current LTS range	Frontend build and tooling
MongoDB	7.x class deployment	Document persistence
Ollama Runtime	Local model runtime	Vision/reasoning generation pipeline
Docker Engine/Compose	Docker 24+ recommended	Containerized orchestration
Browser	Chromium/Firefox class	Frontend access and UAT execution

Table 3.4: Software Environment Requirements

3.7 Constraints

Constraints in this project are not only technical limits but also governance and usability boundaries that directly influence system behavior. The platform must operate under security, role-separation, and infrastructure limitations common in civic environments, while still maintaining user clarity for citizens and workflow efficiency for officials. The following constraint groups define these implementation boundaries and informed several architecture and UI decisions.

3.7.1 UI Constraints

UI constraints define the rendering, state-management, and responsiveness requirements that guard against poor citizen experience in adverse conditions:

- Role-specific pages must enforce access boundaries and clear redirection behavior.
- Analyze/result screens must preserve draft continuity during refresh and error conditions.
- Dashboard cards and maps must remain usable at mobile widths (e.g., 375 px baseline tests).

3.7.2 Communication Interface

The platform communicates exclusively over authenticated REST endpoints; the following interface rules govern how clients and services exchange data:

- REST over HTTP(S) is the canonical communication model.
- Protected routes require authenticated cookie session or compatibility bearer token.
- Production frontend proxy must route API traffic to backend service with /api/v1 compatibility.

3.7.3 Hardware Interface

Hardware interface constraints bound the AI runtime's operational envelope and determine how the system must gracefully degrade under resource pressure:

- AI response times depend on model runtime health and available VRAM.
- GPU contention and model load/unload latency can affect throughput.
- Large image uploads require strict file-size and decode constraints to avoid resource exhaustion.

3.7.4 Criticality of Application

The system is governance-sensitive. Incorrect status transitions, misrouted complaints, or unauthorized updates can directly impact citizen trust and administrative accountability. Hence, status history and role checks are non-negotiable controls.

3.7.5 Safety and Security Considerations

Because civic complaints can contain sensitive personal information, the following security constraints are treated as non-negotiable throughout the codebase:

- Input sanitization is required for free-text fields.
- File intake must detect extension-header mismatches and corrupted payloads.
- Token reset flow must avoid account enumeration and raw token persistence.
- Security headers and CORS policies must remain active on both normal and error responses.

3.8 Assumptions and Dependencies

ID	Assumption/Dependency	Implication
A1	MongoDB instance is reachable and healthy	Core read/write workflows remain functional
A2	Ollama runtime is available with configured models	Analyze and generation pathways remain active
A3	Environment variables are configured accurately	Auth, DB, CORS, and model endpoints behave as intended
A4	Frontend and backend URL mapping is consistent	API calls and route protection remain stable
A5	Worker profiles include valid department and status metadata	Assignment service can select candidates correctly
A6	Deployment host supports Docker where container mode is used	Production-style stack can be reproduced

Table 3.5: Assumptions and External Dependencies

3.9 Detailed Use-Case Narratives

Table 3.6: Narrative Use-Case Requirements

UC ID	Actor	Main Flow	Alternate/Exception Flow
UC-01	Citizen	Register account, login, access dashboard	Duplicate username/email is rejected with validation message
UC-02	Citizen	Upload image and run analyze pipeline	Unsupported/corrupt image rejected with explicit reason
UC-03	Citizen	Edit draft and submit complaint	If analyze unavailable, user submits manual complaint text after retry guidance
UC-04	Citizen	Track status and receive notifications	If no notifications available, panel remains stable with unread count zero

UC ID	Actor	Main Flow	Alternate/Exception Flow
UC-05	Citizen	Submit feedback after complaint resolution	Duplicate feedback attempt blocked with conflict response
UC-06	Worker	View assigned complaints and update availability	Worker pending approval cannot access worker-only operations
UC-07	Worker	Mark complaint done	If assignment mismatch occurs, operation is denied by role/ownership checks
UC-08	Dept Head	Filter department complaints and update status	Dept-head cannot update complaints outside own department
UC-09	Dept Head	Add notes and transfer/escalate complaint	Invalid complaint ID or missing target yields controlled validation error
UC-10	Admin	Approve/reject worker registrations	Already-approved/rejected states handled idempotently
UC-11	Admin	Operate triage queue and analytics	Empty triage queue still returns successful response
UC-12	Public User	View anonymized complaint board	No authenticated personal fields are exposed in response payload

3.10 Requirement Priority Classification

Requirements are classified using the MoSCoW methodology [20] to ensure delivery focus.

Priority	Requirement IDs	Interpretation
Must Have	FR-01 to FR-12, FR-14, FR-16, FR-21	Core operational viability: authentication, complaint life-cycle, role control, assignment, notifications, and service health
Should Have	FR-13, FR-15, FR-17, FR-18, FR-22	Strong governance and maintainability improvements that meaningfully increase institutional readiness
Could Have	FR-19, FR-20	Non-blocking but high-value transparency and reporting improvements
Non-Functional Critical	NFR-01, NFR-02, NFR-06, NFR-10	Security, reliability, observability, and data integrity controls required for trusted operation

Table 3.7: MoSCoW-Style Requirement Priority Mapping

3.11 Chapter Summary

The requirement study establishes the transition from a manually intensive grievance flow to a structured, secure, and auditable digital workflow. Functional and non-functional requirements were formalized in implementation-aligned terms and mapped against realistic user roles and operational constraints.

Chapter 4

Proposed System Requirements

This chapter describes the proposed Jan-Sunwai AI solution architecture from a requirement realization perspective. While Chapter 3 documented what is required, this chapter explains how the proposed system addresses those requirements through modular capabilities and integrated workflows built on the React, FastAPI, MongoDB, and Ollama stack [12, 11, 13, 17].

4.1 Overview of Proposed System

The proposed system is a role-driven civic complaint platform that combines AI-assisted intake with deterministic governance controls. Rather than adopting a monolithic architecture or a purely generative AI approach, the system was designed with strict module boundaries, a hybrid inference pipeline, and a flexible document-store database. These architectural choices were made to ensure the platform could run efficiently in resource-constrained municipal environments while maintaining strict auditability and data residency [11, 13, 7].

The system can be summarized as a five-stage path:

1. Complaint Intake: Citizen uploads image and requests analysis.
2. AI Enrichment: System extracts likely category, location hints, and draft text.
3. Structured Submission: Citizen reviews/edits and confirms complaint payload.
4. Operational Routing: Complaint is assigned, tracked, and escalated if needed.
5. Governance and Transparency: Notifications, analytics, exports, and public feed support accountability.

4.2 Module Descriptions

The proposed system is organized as cooperating modules so that each operational responsibility remains isolated, testable, and maintainable. This modular decomposition also supports phased enhancement, where future upgrades can be applied to specific capability areas without destabilizing the entire platform. The subsections below summarize each major module and the role it plays in requirement realization.

4.2.1 Authentication and User Management Module

This module handles registration, login, session identity, profile updates, and password recovery. It enforces role-oriented authorization for citizen, worker, department head, and admin personas. Key controls include secure cookie-session issuance [10], role checks, JWT compatibility for API clients, and reset-token hashing.

4.2.2 Analyze and Complaint Intake Module

This module processes uploaded images, invokes the hybrid classifier pipeline, extracts location metadata, and orchestrates draft generation. It supports both immediate and queued generation paths and returns timing metadata for observability.

4.2.3 Complaint Lifecycle Module

This module persists complaints and manages state transitions, including status updates, transfer, escalation, feedback, notes, and comments. It appends status history entries for each transition to preserve decision traceability.

4.2.4 Assignment and Worker Operations Module

This module auto-assigns complaints to eligible workers using department matching, optional geospatial service-area checks, and load balancing by active tasks. It also provides worker status controls and completion signaling.

4.2.5 Notification and Communication Module

This module creates in-app notifications for complaint lifecycle events and maintains unread counters. It includes an email relay stub for deployment environments where SMTP integration is required.

4.2.6 Triage and Governance Module

This module supports admin review of low-confidence classifications. It enables decision actions such as approve, reject, or relabel. The triage path acts as a governance safety net against AI uncertainty.

4.2.7 Analytics and Public Transparency Module

This module aggregates operational statistics, trend lines, and geospatial heatmap data. It also exposes an anonymized public feed to support transparency without leaking sensitive user data.

4.3 System Features

Table 4.1: Proposed Feature Set and Operational Benefit

Feature ID	Feature	Operational Benefit
PF-01	Image-first intake	Reduces textual burden on citizen and improves evidence quality
PF-02	Hybrid category detection	Improves classification consistency through rules plus selective reasoning

Feature ID	Feature	Operational Benefit
PF-03	Draft complaint generation	Standardizes language quality and reduces submission ambiguity
PF-04	Role-based dashboard routing	Ensures each persona sees relevant actions and data
PF-05	Auto-assignment service	Reduces manual workload and accelerates field execution
PF-06	Status history timeline	Improves accountability and supports audit and grievance review
PF-07	Notification chain	Keeps citizens informed about progress without repeated manual follow-up
PF-08	Triage queue	Prevents low-confidence AI outputs from silently propagating
PF-09	Bulk admin actions	Improves throughput for backlog management in peak periods
PF-10	Heatmap and analytics	Supports evidence-driven supervision and resource planning
PF-11	Public complaint feed	Increases transparency and trust in civic workflow visibility
PF-12	Health endpoints and timing logs	Supports operational diagnosis and production monitoring

4.4 Advantages of Proposed System

The proposed solution offers measurable value across three levels: citizen experience, administrative throughput, and institutional governance. Crucially, the chosen architecture (FastAPI + React + MongoDB + Local LLM) outperforms simpler monolithic alternatives in several ways. The document model (MongoDB) is better suited to civic records, which evolve through complex, schema-less state transitions (such as appending notes and timestamps) without requiring expensive relational joins. Furthermore, the hybrid AI approach ensures that the non-deterministic nature of vision models is bounded by hard-coded, deterministic routing SLAs, making the system vastly more reliable for public service than a pure ”black box” generative classifier.

4.4.1 Citizen-Centric Advantages

The citizen-facing advantages below stem directly from the image-first intake and AI-assisted drafting architecture:

- Simplified complaint filing with image-based assistance.
- Better complaint quality due to generated formal drafts.
- Real-time feedback through status and notification channels.
- Reduced confusion in choosing the correct municipal department.

4.4.2 Administrative and Departmental Advantages

On the administrative side, the module boundaries and assignment automation translate into measurable operational gains:

- Better routing quality and lower manual triage burden for standard cases.
- Strong lifecycle auditability for supervisory and escalation actions.
- Faster assignment and backlog control through automation and bulk operations.
- Structured data support for reporting and policy-level analysis.

4.4.3 Institutional Advantages

At the institutional level, the deployment architecture delivers compliance and handover advantages not possible with ad-hoc cloud-dependent designs:

- Local-first inference supports stricter data residency requirements.
- Containerized deployment supports repeatable setup and handover.
- Explicit API version aliasing improves integration stability.
- Security hardening and validation controls reduce common exploit surfaces.

4.5 Requirement Realization Matrix

Table 4.2: Traceability from Requirements to Proposed Modules

Requirement Group	Primary Module	Realization Mechanism
Registration and Login	Auth and User Management	Register/login routes, cookie-session issuance, JWT compatibility mode, profile update API, password reset token lifecycle
AI Classification and Drafting	Analyze Module + LLM Queue	Vision model cascade, rule engine scoring, optional reasoning, queued generation polling

Requirement Group	Primary Module	Realization Mechanism
Lifecycle and Tracking	Complaint Lifecycle Module	CRUD plus status, transfer, escalate, feedback, notes, comments, status history
Worker Operations	Assignment and Worker Module	Eligibility filter, geospatial check, load balancing, worker completion flow
Governance and Review	Triage and Admin Controls	Low-confidence review queue and corrective decision actions
Communication	Notifications Module	In-app notification events, unread counter, mark-read APIs, email stub integration
Reporting and Transparency	Analytics/Public Module	Overview metrics, heatmap endpoint, anonymized public complaint feed
Resilience and Security	Cross-cutting Platform Controls	Health probes, security headers, upload hardening, sanitization, rate limits, logs

4.6 Proposed Operational Workflow

Proposed End-to-End Workflow

Citizen submits evidence, AI enriches payload, complaint is stored and routed, worker and department users execute actions, and governance views aggregate operational signals.

Figure 4.1: Proposed End-to-End Workflow

4.7 Expected Impact

The platform is expected to produce measurable improvements across six dimensions. Complaint quality will improve as guided image intake and AI drafting reduce inconsistent descriptions. Routing efficiency will increase as department prediction and authority mapping eliminate the first-hop re-assignment delay. Resolution governance becomes stronger through status timelines, notes, and escalation controls. Citizen trust improves through notifications and trans-

parent progress signals. Operational analytics give administrators better planning tools through department trends and heatmap indicators. Finally, deployment readiness is lowered through a unified compose and environment strategy.

4.8 Module Interaction Contracts

Table 4.3: Inter-Module Contract Summary

Producer Module	Consumer Module	Interaction Contract
Authentication module	All protected routers	Provides current user identity and role claims via dependency injection
Analyze module	Complaint lifecycle module	Produces classification/location/draft payload used for complaint creation
Complaint lifecycle module	Notification module	Emits status/registration/escalation events for user-facing notifications
Complaint lifecycle module	Assignment module	Triggers auto-assignment at complaint creation and slot-free reassignments
Worker module	Complaint lifecycle module	Reports completion updates that influence lifecycle and status history
Triage module	Complaint lifecycle module	Applies corrected decisions for low-confidence complaint records
Analytics module	Complaint data store	Executes aggregate queries for dashboard and heatmap responses
Public module	Complaint data store	Exposes anonymized subset for transparency without privileged fields
Health module	Database/model runtime	Publishes liveness/readiness/dependency status for operations

4.9 Requirement Realization Detail by Persona

Each persona receives a targeted feature set. **Citizens** get image-based intake, dashboard tracking, notifications, feedback, and password reset — reducing filing friction and improving visibility. **Workers** receive an assignment list, status update controls, and completion actions — delivering structured field execution. **Department Heads** have department-level filtering, status updates, transfer/escalation, and notes — enabling controlled supervisory governance. **Administrators** hold full operational governance: worker approvals, triage decisions, bulk operations,

analytics, and exports. **Public Viewers** can access the anonymised complaint board without exposing any personally identifiable records.

4.10 Proposed Data and Control Flow Narrative

The proposed flow intentionally separates decision support from governance authority. AI modules suggest categories and draft text, but acceptance and lifecycle transitions remain controlled through authenticated role actions. This hybrid control plane avoids over-automation risk while still reducing manual burden in high-volume intake scenarios.

4.11 Chapter Summary

The proposed system translates requirement intent into integrated modules that jointly address user friction, operational traceability, and governance accountability. It deliberately combines AI assistance with deterministic controls and administrative safety nets, making it suitable for practical municipal deployment contexts.

Chapter 5

System Design

System design in Jan-Sunwai AI focuses on reliable complaint orchestration in a governance-sensitive environment. The design balances modular API responsibilities, AI integration boundaries, role-specific UI behavior, and document-oriented persistence strategies.

5.1 System Architecture Design

System architecture design was developed to ensure clear separation of concerns while preserving end-to-end traceability across complaint intake, processing, and governance review. The design prioritizes predictable integration boundaries among frontend, backend, data, and AI runtime layers so that failures can be isolated and recovered without cascading disruption. This architecture also supports future scaling through modular service evolution [11, 12, 13, 17, 16].

5.1.1 Architectural Style

The platform follows a rigorous **Client-Server architecture** with a layered service approach:

- **Presentation Layer**: React SPA (using React Router v7 and Vite) with role-aware route protection and Tailwind v4 styling.
- **API Layer**: FastAPI routers strictly enforced under the `/api/v1` namespace grouped by domain concern (Complaints, Users, Health, Triage, Analytics, Workers).
- **Service Layer**: In-memory async AI Queues, Automated Worker Assignment, sanitization grids, robust Rate Limiting (`slowapi`), and background escalation loops (`escalation_loop()`).
- **Data Layer**: Primary MongoDB collections (with `2dsphere` indexes) alongside a dedicated NDMC Audit Database.
- **AI Runtime Layer**: Ollama-hosted local models utilizing a vision classification pipeline and a reasoning fallback.

Overall System Context Diagram

Users interact with the frontend, which communicates with backend APIs. Backend coordinates MongoDB, uploads storage, AI runtime, notification logic, and escalation loop.

Figure 5.1: Overall System Context Diagram

Component Architecture Diagram

Frontend modules map to backend routers and services. Persistence collections and model runtime are isolated behind service interfaces for maintainability and testability.

Figure 5.2: Component Architecture Diagram

5.1.2 Key Architectural Decisions

Key design decisions are summarized below:

Decision	Rationale
Hybrid AI pipeline (vision + rules + optional reasoning)	Reduces hallucination risk, lowers VRAM pressure, and improves deterministic routing for common civic scenes
Document model in MongoDB	Supports rich complaint lifecycle payloads and embedded audit metadata without heavy relational join overhead
Role-specific route gates	Enforces operational boundaries between citizen, worker, department head, and admin personas
Queue-based draft generation fallback	Prevents long-running model calls from blocking frontend experience
Security middleware and sanitization by default	Reduces exploit surface for API consumers and file uploads
Versioned API alias (/api/v1)	Improves long-term integration stability for external municipal consumers

Table 5.1: Major Design Decisions and Rationale

5.1.3 AI Classification Pipeline

To ensure reliable routing, the AI component employs a hybrid model combining vision-language inference with deterministic thresholding. When an image is uploaded, the vision model generates class probabilities. The final confidence score is formulated as the maximum probability across predefined civic categories:

$$\text{Confidence} = \max_i(P(\text{class}_i \mid \text{image})) \tag{5.1}$$

If this Confidence exceeds the system's acceptable threshold (e.g., 0.70), the complaint is auto-routed to the corresponding department. If the confidence falls below this threshold, the system flags the submission for the administrative triage queue, ensuring human oversight handles ambiguous cases.

5.2 UML Diagrams

The following diagrams are aligned with the implemented module boundaries, routing flow, and lifecycle stages documented in the project report and API references.

5.2.1 E-R Diagram

E-R Diagram

Primary entities: User, Complaint, Notification, and PasswordReset token.
Relationships include complaint ownership, assignment, and notification generation.

Figure 5.3: E-R Diagram

Data Flow & Rationale: This diagram shows how complaint records act as the central nexus linking users, workers, and notifications. By embedding status history directly within the Complaint entity, the database minimizes costly relational joins during dashboard rendering.

5.2.2 Use Case Diagram

Use Case Diagram

Citizen submits and tracks complaints; worker resolves assignments; department head updates status; admin governs approvals, triage, and analytics.

Figure 5.4: Use Case Diagram

Role Segregation & Governance: This diagram captures the strict access boundaries enforced by the system. While citizens have broad permission to create and track records, state mutations (such as resolving or rejecting a complaint) are heavily restricted to authenticated workers and department heads. It visually demonstrates the governance principle that AI assists but humans hold final approval authority.

5.2.3 Class Diagram

Class Diagram

Schema-level classes represent User, Complaint, and Notification with association to role, status, and metadata properties.

Figure 5.5: Class Diagram

Data Encapsulation: The class structure reflects the document-oriented database design where complex types like Geolocation and Timestamps are embedded rather than relationally linked. This tradeoff sacrifices some normalization but ensures that fetching a single Complaint object provides the frontend with the complete operational context needed to render a dashboard.

5.2.4 Sequence Diagram

Sequence Diagram

Upload -> analyze -> generate draft -> submit complaint -> auto-assign -> status updates -> notifications.

Figure 5.6: Sequence Diagram

Flow & Bottlenecks: This sequence outlines the chronological journey of a grievance. It highlights the AI inference step as the primary synchronous bottleneck. To prevent browser timeouts, the system restricts the analysis time window and degrades gracefully to a manual form if inference times out.

5.2.5 Activity Diagram

Activity Diagram

Lifecycle from citizen intake through operational processing and closure with feedback and audit logging.

Figure 5.7: Activity Diagram

State Machine Constraints: This diagram tracks the rigid state transitions a complaint undergoes (e.g., from ‘Pending‘ to ‘In Progress‘ to ‘Resolved‘). It illustrates the system’s fail-safes: if the AI classification confidence is low, the activity branches into the administrative triage queue rather than failing silently, preventing black-hole scenarios in civic reporting.

5.2.6 DFD Diagram

Data Flow Diagram

Data moves from frontend to API services, then to persistence and AI runtime. Notification and analytics streams derive from lifecycle events.

Figure 5.8: Data Flow Diagram

System Dynamics: The DFD maps the transition of unstructured citizen uploads into structured database records. It illustrates why the AI module is isolated from the core CRUD API—allowing the transaction engine to operate independently of the compute-heavy vision processing.

5.2.7 Deployment Diagram

Deployment Diagram

Browser clients access frontend service; frontend proxies API calls to backend service; backend communicates with MongoDB and host-level Ollama runtime.

Figure 5.9: Deployment Diagram

Local-First Architecture: The deployment topology explicitly avoids managed cloud dependencies to comply with municipal security constraints. By keeping the React bundle, FastAPI server, MongoDB, and Ollama LLM isolated within a Docker Compose network, the diagram proves that the entire grievance platform can be air-gapped or run on constrained on-premises infrastructure.

5.3 Database Design

Database design focused on lifecycle integrity, query efficiency, and auditability rather than strict relational normalization. Since complaint records evolve through multiple state transitions and actor interactions, the chosen document model keeps operationally related history close to primary complaint entities while retaining key references for role and ownership controls. This approach improves retrieval speed for dashboards, timelines, and governance reviews [13, 11, 14].

5.3.1 Table Design and Relationships

The database is organized around four principal collections:

- **users**: identity, role, and worker service-area metadata.
- **complaints**: complaint payload, routing metadata, status history, and collaboration artifacts.
- **notifications**: user-scoped notification events and read-state tracking.
- **password_resets**: token hash lifecycle with expiry and usage state.

The design intentionally keeps lifecycle and audit structures close to complaint documents for locality and retrieval simplicity.

5.3.2 Normalization

The platform uses controlled denormalization:

- Embedded `status_history`, `dept_notes`, and `comments` reduce query complexity.
- Core identity references (e.g., `user_id`, `assigned_to`) remain external for role and profile updates.
- Routing metadata is persisted with complaint records to preserve deterministic decision context over time.

This approach preserves operational read performance while retaining clear ownership semantics.

5.3.3 Indexing Strategy

Key indexing strategies are summarized below:

Collection	Index	Purpose
users	unique(username), unique(email)	Fast auth lookups and uniqueness enforcement
complaints	(user_id, created_at)	Efficient citizen dashboard history retrieval
complaints	(status, created_at)	Queue and status-oriented admin filtering
complaints	(department, created_at)	Department-level operational listing
complaints	authority_id	Escalation and authority review queries
notifications	(user_id, created_at)	Ordered notification feed retrieval
notifications	(user_id, is_read)	Unread badge counter queries
password_resets	unique(token_hash)	Token lookup and collision prevention
password_resets	(user_id, used)	Active token invalidation management
password_resets	TTL(expires_at)	Automatic expired-token cleanup

Table 5.2: Indexing Strategy by Collection

5.4 GUI Design

GUI design was planned as a task-oriented interface model where each role sees only the controls and data required for its responsibilities. Visual hierarchy, workflow clarity, and responsive behavior were treated as core functional requirements because field and supervisory users interact with the platform under different device and time constraints. The resulting interface strategy emphasizes speed of action, low ambiguity, and consistent feedback.

5.4.1 Design Principles

The UI follows practical governance portal principles:

- Clarity of action over decorative complexity.
- Explicit role boundaries in navigation and controls.
- Immediate visual feedback for status and notification changes.
- Responsive behavior for desktop operator screens and mobile field usage [21].

5.4.2 Interface Areas

Key interface areas are summarized below:

Interface Area	Purpose
Public Home	Awareness, role entry points, and transparency access
Authentication Screens	Login, registration, forgot password, reset password
Analyze and Result Views	Upload, AI analysis display, map interaction, draft review
Citizen Dashboard	Complaint list, timeline, feedback, and progress visibility
Worker Dashboard	Assigned queue and completion updates
Department Head Dashboard	Department-level filtering, status updates, and notes
Admin Dashboard	Global queue, approvals, triage, bulk actions, analytics, exports
Notification Panel	Event feed and read-state management
Profile Screen	Personal information update controls

Table 5.3: GUI Areas and Purpose

5.4.3 Wireframe Reference

The following figure provides a visual reference that complements the explanatory text in this subsection.

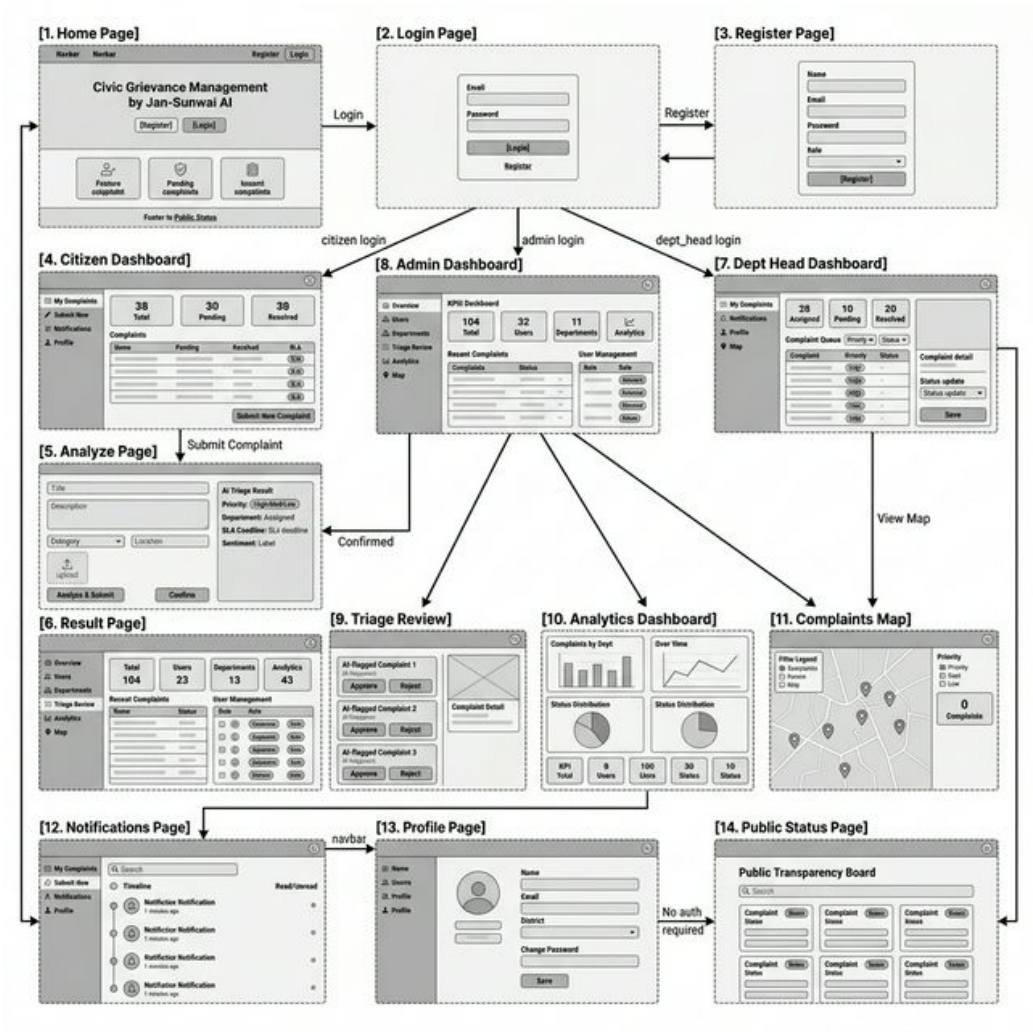
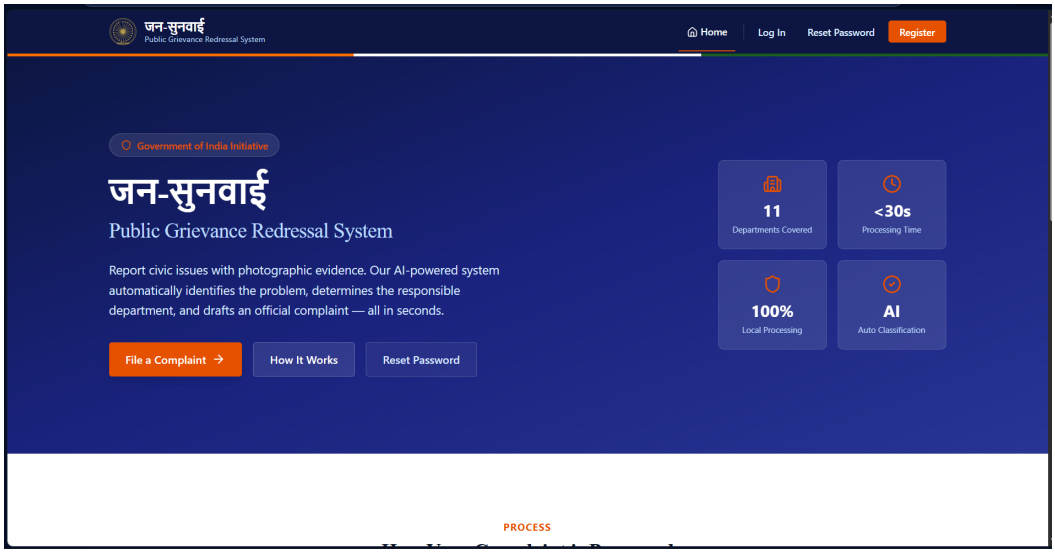


Figure 5.10: GUI Wireframe Reference from Project Documentation

5.5 Screenshots

The screenshot gallery below restores representative evidence from the implementation while keeping the report compact enough for submission.



Screenshot 5.1: Home Page Hero and Public Entry CTA

Screenshot 02
AI Analysis Result

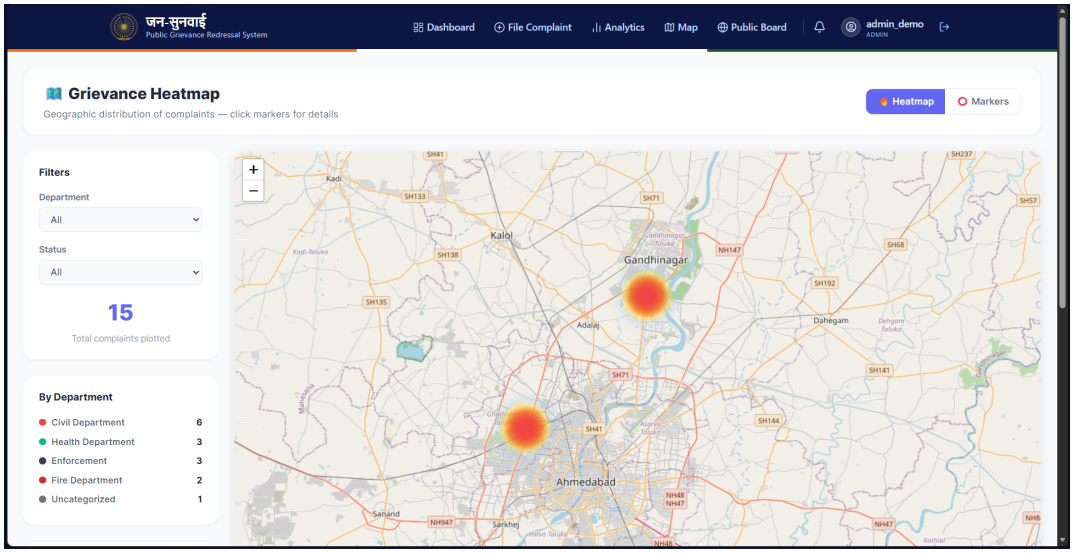
Screenshot 5.2: AI Analysis Result and Draft Review

Screenshot 03
Citizen Dashboard

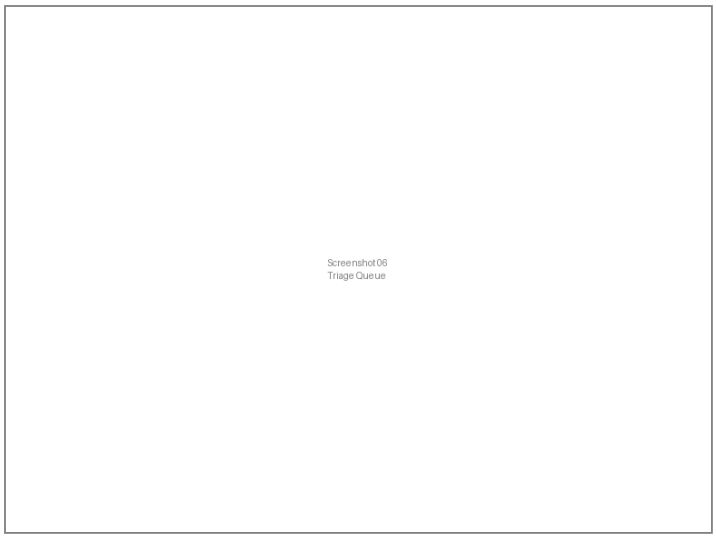
Screenshot 5.3: Citizen Complaint Tracking Dashboard

Screenshot 04
Worker Dashboard

Screenshot 5.4: Worker Assignment Queue



Screenshot 5.5: Admin Dashboard — shows real-time complaint volume, department breakdown, and pending approvals. The admin can initiate bulk status updates or export records from this single view.



Screenshot 5.6: Administrative Triage Queue — displays low-confidence AI classifications awaiting human review. The confidence score, AI-suggested category, and uploaded image are shown side-by-side so the admin can approve, reject, or relabel in one action.

5.6 Chapter Summary

This chapter defined structural design choices, module interactions, data modeling strategy, and UI architecture. The design emphasizes traceability, secure operations, and practical maintainability while accommodating AI-assisted workflows in a municipal governance context.

Chapter 6

Implementation Planning

Implementation planning for Jan-Sunwai AI focused on three parallel goals: reproducibility, maintainable module growth, and deployment readiness. The implementation strategy treated backend, frontend, AI runtime, and operational scripts as coordinated components rather than isolated code tracks.

6.1 Implementation Environment

The implementation environment was designed to keep developer productivity high while preserving deployment realism. To achieve this, the project supports both fast local iteration and production-style container verification with aligned configurations, health checks, and service contracts. This dual-mode environment reduces environment drift and ensures that behavior validated during development remains reliable during handover and deployment [16, 11, 12, 13, 17].

6.1.1 Local Development Environment

The local workflow supports rapid iteration with role-based UI validation and backend testing. Typical development topology includes:

- Backend running on FastAPI/Uvicorn at port 8000.
- Frontend development server on Vite at port 5173.
- MongoDB either local service or containerized dependency.
- Ollama runtime with configured model set on host machine.

The local setup was selected to reduce iteration latency while preserving fidelity with production behavior through shared APIs, environment variables, and health semantics [16, 11, 17].

6.1.2 Production-Style Container Environment

For deployment simulation and handover readiness, the project includes a unified compose configuration with a production profile [16]. In production mode:

- MongoDB runs as a dedicated service with health checks.
- Backend runs from production Dockerfile and mounts uploads volume to `/app/uploads`.
- Frontend runs in Nginx container under compose profile `prod`.
- Health checks and log rotation policies are explicitly configured.

This ensures that local verification and deployment handover use the same operational assumptions.

6.2 Tools & Technologies Used

Table 6.1: Implementation Toolchain and Use Cases

Category	Tool/Library	Implementation Use
Backend Framework	FastAPI	Route definitions, dependency injection, validation, middleware pipeline
Data Validation	Pydantic [15]	Request/response schema validation and structured models
Auth and Security	python-jose, passlib[bcrypt]	Cookie-session auth, JWT compatibility support, and password hashing
HTTP Testing	httpx	Async API smoke and integration checks
Database Driver	Motor/PyMongo stack	MongoDB async operations and collection access
AI Runtime	Ollama	Local vision and reasoning model invocation
Image Handling	Pillow	Upload payload validation and EXIF processing
Frontend Framework	React	Role-aware SPA component and page architecture
Build Tool	Vite	Fast frontend development and optimized production build
Client HTTP	Axios	Frontend API communication layer
Routing	react-router-dom	Protected routes and role-based navigation
Containerization	Docker + Compose	Reproducible orchestration of backend/frontend/database services
QA Automation	pytest, locust	Unit/integration/security/resilience/load test execution
Documentation	Markdown + LaTeX	Runbooks, architecture docs, and final academic report

6.3 Key Implementation Patterns

Three representative code patterns are presented here to demonstrate that the toolchain described above was applied concretely. These are taken directly from the production codebase and illustrate critical implementation decisions.

6.3.1 Upload Validation Chain

Every image submitted by a citizen passes through a layered validation chain before it is persisted or forwarded to the AI runtime. The following pattern, implemented in `services/storage.py`, checks file extension against the actual binary signature to prevent MIME-type spoofing attacks:

```
ALLOWED_SIGNATURES = {
    b"\xff\xd8\xff": "jpg",
    b"\x89PNG\r\n": "png",
    b"GIF8": "gif",
    b"RIFF": "webp",
}

def validate_image(file_bytes: bytes, filename: str) -> str:
    header = file_bytes[:6]
    for sig, ext in ALLOWED_SIGNATURES.items():
        if header.startswith(sig):
            return ext
    raise HTTPException(status_code=400,
        detail="File content does not match declared type")
```

6.3.2 Classifier Confidence Thresholding

The hybrid classification pipeline routes the complaint automatically only when model confidence exceeds a configurable threshold. Below it, the complaint is sent to the administrative triage queue. This logic resides in `services/classifier.py`:

```
CONFIDENCE_THRESHOLD = 0.70

def route_or_triage(result: ClassifierResult) -> TriageDecision:
    if result.confidence >= CONFIDENCE_THRESHOLD:
        return TriageDecision(
            department=result.department,
            auto_routed=True,
            requires_review=False
        )
    return TriageDecision(
        department=None,
        auto_routed=False,
        requires_review=True,
```

```
        low_confidence_score=result.confidence
    )
```

6.3.3 Frontend Route Guard

Role-based page access is enforced client-side through a `ProtectedRoute` wrapper component. This ensures that navigating directly to an admin URL while authenticated as a citizen triggers an immediate redirect rather than a permission error:

```
const ProtectedRoute = ({ allowedRoles, children }) => {
  const { user, isLoading } = useAuth();
  if (isLoading) return <LoadingSpinner />;
  if (!user) return <Navigate to="/login" replace />;
  if (!allowedRoles.includes(user.role))
    return <Navigate to="/dashboard" replace />;
  return children;
};
```

6.4 Coding Standards Followed

6.4.1 Backend Standards

The backend was implemented with the following structural conventions so that every route, schema, and service remained independently testable:

- Router-based modularization for domain isolation.
- Strict schema validation through typed request models.
- Clear HTTP semantics for validation, auth, and lifecycle errors.
- Service extraction for cross-cutting concerns (assignment, sanitization, storage, email).
- Structured logging and failure-safe response behavior.

6.4.2 Frontend Standards

Frontend code separated concerns so that role-aware pages, shared components, and API communication remained independently updatable:

- Role-aware route guards and dashboard redirection.
- Separation of pages, reusable components, layouts, and context providers.
- Consistent API base usage through environment variables.
- Explicit loading/error states for user-facing long-running operations.
- Responsive design checks for desktop and mobile interaction baselines.

6.4.3 Configuration Standards

Environment and configuration consistency was treated as a first-class engineering concern, governed by these conventions:

- Single canonical backend environment file for consistency.
- Explicit production defaults and overrides for sensitive settings.
- Avoidance of duplicate compose and env artifacts to reduce drift.
- Documented scripts for startup, tests, deployment simulation, and verification.

6.5 Implementation Work Packages

Implementation work was divided into clearly scoped packages so that each delivery stream could be validated independently and then integrated with controlled dependency checks. This packaging approach improved scheduling clarity, reduced merge risk, and made milestone tracking more objective for both technical and management review.

6.5.1 Work Package 1: Core Backend and Data Layer

Deliverables included app bootstrap, database connection lifecycle, auth primitives, user and complaint schemas, and initial complaint CRUD operations. This package established foundational APIs and validation behavior.

6.5.2 Work Package 2: AI Pipeline Integration

Deliverables included classifier orchestration, model fallback strategy, rule-engine scoring, queue-based generation path, and timing metadata in analysis responses. This package transformed basic upload capability into intelligent intake flow.

6.5.3 Work Package 3: Frontend Role Flows

Deliverables included authentication screens, analyze/result pathway, role dashboards, notifications UI, and profile controls. Route guards and dashboard routing ensured role-appropriate user journeys.

6.5.4 Work Package 4: Operational Features

Deliverables included worker assignment logic, timeline/audit features, comments/notes, escalation, triage, analytics, and CSV exports. This package completed department-level and governance-level operational requirements.

6.5.5 Work Package 5: Security and Reliability Hardening

Deliverables included upload integrity checks, sanitization, security headers, CORS tightening, rate-limiting toggles, readiness/liveness checks, graceful AI degradation, and resilience-focused tests.

6.5.6 Work Package 6: Deployment and Handover Readiness

Deliverables included the production compose profile, deployment guides, and API references. The package also included backup scripts and verification procedures. This package enabled transition from a development artifact to a deployable product.

6.6 Implementation Sequence and Dependencies

Step	Implementation Activity	Dependency Consideration
1	Establish backend skeleton and DB connectivity	Required before auth and complaint persistence
2	Build auth/session primitives	Required before protected API and role-specific frontend routes
3	Implement analyze pipeline and storage handling	Required before citizen intake flow can be tested end-to-end
4	Build frontend analyze/result workflow	Depends on stable analyze payload and auth behavior
5	Add assignment and lifecycle controls	Depends on complaint schema maturity and worker metadata
6	Add triage/api/v1/analytics and governance endpoints	Depends on reliable complaint state and metadata richness
7	Add security and reliability hardening	Should follow core flow stability to avoid hidden regressions
8	Run tests and UAT cycles	Depends on complete feature slices and stable environments
9	Prepare production profile and handover docs	Depends on validated security/performance baseline

Table 6.2: Implementation Sequence with Dependency Notes

6.7 Configuration and Secret Planning

The implementation plan emphasized explicit secret handling and runtime configurability. Important control groups include:

- Authentication controls: cookie-session settings, JWT secret/algorithm, token expiry, compatibility mode.
- Data controls: Mongo URL and database selection.
- AI controls: model identifiers, timeout windows, queue worker count.
- Security controls: CORS origin allowlist and rate-limiter mode.
- Communication controls: SMTP host/port/from for notification relay.

A notable planning correction during implementation was moving to one canonical backend

environment file to eliminate drift between local and production templates.

6.8 Quality Gates in Implementation Plan

Gate	Acceptance Criterion
Functional API baseline	Health endpoints and core auth routes return expected responses
Analyze pipeline gate	Valid image upload produces structured classification and draft status output
Lifecycle gate	Complaint create/update/assignment/notification chain works end-to-end
Security gate	Upload hardening, sanitization, role checks, and headers validated in tests
Reliability gate	Graceful fallback behavior under model/DB faults verified
Deployment gate	Compose stack health checks pass and service topology is reproducible
UAT gate	Citizen and admin scripts complete without blocking defects
Documentation gate	Deployment, API, user manual, and report artifacts synchronized

Table 6.3: Implementation Quality Gates

6.9 Resource and Effort Planning

Implementation effort was distributed between development, stabilization, testing, and documentation. Rather than pushing all testing to the end, the plan incorporated recurring verification cycles after major feature groups. This reduced late-stage uncertainty and shortened defect turnaround time.

6.10 Detailed Module Implementation Notes

This section captures implementation-level observations that are useful for maintenance, future enhancement, and institutional handover. It complements high-level planning by documenting how module boundaries, service responsibilities, and operational safeguards were realized in practice.

6.10.1 Backend Router Implementation Notes

Each router group below shows how its implementation responsibilities were planned and what specific engineering decisions arose during delivery:

Table 6.4: Router-Level Implementation Planning Notes

Router Group	Core Responsibilities	Planning/Implementation Considerations
users	registration, login, profile update, password reset	Required generic forgot-password response to prevent account enumeration; reset tokens persisted as hashes with expiry lifecycle
complaints	analyze, create, list, detail, update, transfer, escalate	Needed consistent status-history append behavior and role-aware update guards; analyze route required graceful 503 degrade path
workers	assignment views, approval/rejection, service-area updates	Needed separation between worker self-actions and admin governance actions; assignment diagnostics endpoint added for ops clarity
notifications	list, unread count, mark single/all read	Badge consistency required deterministic unread counter behavior and ownership checks
triage	review queue and decision endpoint	Admin-only corrective path for low-confidence AI cases; decision audit notes required
analytics/api/v1/public	dashboards, heatmap, anonymized public listing	Query aggregation planning required pagination defaults and safe exposure model for public feed
health	live/ready/models/gpu probes	Needed operationally meaningful readiness semantics and low-overhead liveness behavior

6.10.2 Service-Layer Planning Notes

The matrix below details the specific planning components necessary to operationalize this segment of the architecture.

Service	Planning Note
Assignment service	Location-aware proximity filtering with fallback behavior when location metadata is missing
Storage service	Validation pipeline designed as extension + content-type + magic-number + payload decode sequence
Sanitization service	Planned as reusable utility to keep payload cleaning centralized and testable
LLM queue service	Introduced to absorb long-running draft generation and preserve responsive user experience
Escalation loop	Planned as background process with authority parent mapping and timeline updates
Email service	Implemented as environment-driven stub/relay abstraction for gradual production integration

Table 6.5: Service-Level Implementation Planning

6.11 Deployment Runbook and Rollout Plan

The deployment runbook formalizes a stepwise rollout process so that releases remain reproducible, diagnosable, and reversible. It defines readiness checks, smoke validations, and rollback expectations to minimize service disruption during environment transitions.

6.11.1 Rollout Stages

The matrix below details the specific planning components necessary to operationalize this segment of the architecture.

Stage	Objective	Execution Checklist
R1	Environment readiness	Verify host runtime versions, model availability, and environment variables
R2	Service bootstrap	Start MongoDB and backend; validate health/live and health/ready probes
R3	Frontend linkage	Start frontend/proxy path and verify API base routing behavior
R4	Functional smoke	Execute login, analyze, complaint creation, and notification badge checks
R5	Governance smoke	Validate admin triage, worker assignment controls, and export flow
R6	Hardening validation	Execute security/resilience test scripts and capture baseline metrics
R7	Handover sign-off	Archive logs, backups, and runbook evidence for deployment package

Table 6.6: Deployment Rollout Stages

6.11.2 Rollback Planning

Rollback planning was included to avoid high-impact downtime during deployment changes:

- Preserve previous container images and compose configuration snapshots.
- Ensure database backup is taken before schema/index-affecting changes.
- Revert frontend image and backend service independently when only one tier regresses.
- Maintain known-good environment variable snapshot for rapid restoration.

6.12 Operational Monitoring Plan

Table 6.7: Operational Monitoring Matrix

Signal	Collection Source	Operational Interpretation
API process time	Response header `X-Process-Time`	Detect slow endpoints and regressions after deployments
Readiness status	/api/v1/health/ready endpoint	Indicates dependency-level degradation, mainly database health

Signal	Collection Source	Operational Interpretation
Model availability	/api/v1/health/models endpoint	Confirms runtime model inventory and inference readiness
GPU/model state	/api/v1/health/gpu endpoint	Indicates active model and VRAM-related load behavior
Notification backlog	unread-count behavior and list volume	Detects delayed event propagation or read-state anomalies
Assignment queue growth	open/unassigned complaints and worker status	Helps identify staffing bottlenecks and area mismatch
Triage queue size	low-confidence complaint count	Indicates classifier uncertainty trend and review load
Error logs	rotating backend logs	Captures fault signatures for rapid remediation

6.13 Change Management and Release Strategy

Change management was designed to ensure that feature updates do not weaken stability, security, or governance controls. Each release cycle therefore combines technical verification, documentation synchronization, and rollback preparedness before production-oriented sign-off.

6.13.1 Release Checklist

The sequence below presents the recommended order of actions and priorities for this subsection.

1. Confirm automated tests pass for backend and frontend quality gates.
2. Verify compose validation for default and production profiles.
3. Validate security controls: headers, CORS allowlist, upload checks, and role restrictions.
4. Confirm migration/index scripts produce expected database state.
5. Review documentation deltas (API, deployment, report references) before tagging release.
6. Capture release notes with risk statements and rollback pointers.

6.13.2 Implementation Learnings

Implementation planning revealed that operational simplification tasks (for example, consolidating environment and compose artifacts) can have impact comparable to feature work. These tasks reduced recurring integration errors, made CI behavior more predictable, and improved deployment communication across stakeholders.

6.14 Chapter Summary

Implementation planning translated conceptual design into execution-ready work packages, sequence constraints, and quality gates. The plan successfully balanced rapid feature delivery with operational robustness, resulting in a deployable and documentable civic platform.

Chapter 7

Testing

Testing for Jan-Sunwai AI was designed as a layered strategy combining automated checks, manual UAT scripts, resilience drills, and security/load validation. Because the platform supports governance-sensitive workflows, testing emphasized not only feature correctness but also operational integrity under degraded conditions [18, 7, 8].

7.1 Testing Plan

The testing plan was structured to validate correctness, security, and operational resilience across the full grievance lifecycle. Instead of relying on a single test phase, verification activities were layered and repeated at key implementation checkpoints to reduce late-stage defects.

7.1.1 Testing Objectives

Testing objectives were scoped to cover the distinct failure modes that a production civic system must defend against:

- Validate correctness of core APIs and role authorization boundaries.
- Verify AI-path behavior for valid, invalid, and failure scenarios.
- Confirm notification chain consistency across complaint lifecycle changes.
- Validate resilience behavior under model/database disruptions.
- Ensure production-readiness through integration, load, and UAT checks.

7.1.2 Test Environment

The testing environment baseline outlined below ensures consistency across all validation phases.

Environment Aspect	Configuration
Backend runtime	FastAPI application with test clients and dependency overrides
Database mode	MongoDB-backed flows with selective mock/fake collection tests
Frontend mode	Vite build/lint validation and persona-driven manual walk-throughs
Model runtime mode	Ollama available for normal flows; manually disabled for resilience checks
Automation tools	pytest, httpx, locust, script wrappers for platform-specific execution

Table 7.1: Testing Environment Baseline

7.1.3 Coverage Matrix

The coverage matrix below maps each test suite to its type and the specific system behaviours it validates:

Table 7.2: Testing Coverage Matrix by Suite

Test Artifact	Type	Coverage Highlights
`test_schemas.py`	Unit Validation	GeoLocation bounds, complaint schema validity, confidence-score guardrails
`test_auth_permissions.py`	Unit Authorization	Token claims, admin and worker/dept-head access guards
`test_api_integration.py`	Integration Smoke	Root route, health checks, versioned aliases, lightweight CI path
`test_api_matrix.py`	Integration Matrix	/api/v1 profile route, CORS preflight behavior, security header presence
`test_notification_chain.py`	Integration Behavior	Status update notification creation, email hook path, mark-all-read logic

Test Artifact	Type	Coverage Highlights
`test_resilience_security.py`	Resilience + Security	Upload hardening, analyze degrade 503, readiness degradation, sanitization
Manual UAT scripts	User Acceptance	Citizen and admin journeys, friction logging, UX verification
Load scripts (locust)	Performance	Endpoint mix throughput and graceful degradation behavior

7.2 Types of Testing

Multiple testing types were applied because the platform combines API logic, role-sensitive workflows, AI-assisted features, and deployment constraints. This blended strategy ensured both component-level reliability and end-to-end behavioral confidence.

7.2.1 Unit Testing

Unit tests focused on isolated validation and control logic.

Case ID	Area	Expected Behavior
UT-01	GeoLocation bounds	Reject latitude outside $[-90, 90]$ and longitude outside $[-180, 180]$
UT-02	AI confidence schema	Reject confidence values outside $[0, 1]$
UT-03	Admin role check	Non-admin caller receives HTTP 403 on admin-only dependency
UT-04	Worker approval gate	Unapproved worker cannot pass worker-only dependency path
UT-05	Sanitization function	Script payload transformed to escaped safe representation

Table 7.3: Representative Unit Test Cases

7.2.2 Integration Testing

Integration tests verified route composition, middleware behavior, and cross-router guarantees such as versioned aliases and headers.

- Root and liveness endpoints confirmed service online state.

- Versioned alias endpoints under `/api/v1` validated integration compatibility.
- Security headers confirmed on endpoint responses.
- Preflight CORS behavior validated for allowed origins.

7.2.3 System Testing

System testing executed end-to-end flows across authentication, analyze, complaint lifecycle, assignment, and notification propagation. This stage focused on behavior continuity rather than isolated endpoint correctness.

Scenario	Workflow	Pass Condition
ST-01	Citizen upload -> analyze -> submit	Complaint record created with status history initialized
ST-02	Dept-head status update	Citizen receives notification and timeline reflects transition
ST-03	Worker marks complaint done	Complaint transitions and worker slot logic updates as expected
ST-04	Admin triage decision	Ambiguous complaint receives governed label decision path
ST-05	CSV export	Export file generated with expected schema columns

Table 7.4: System Test Scenarios

7.2.4 User Acceptance Testing (UAT)

The UAT plan was executed through persona scripts:

- Citizen script: login, analyze, map review, submit, dashboard verification.
- Admin script: queue filtering, bulk actions, heatmap view, reassignment, CSV export.
- Feedback loop: friction capture with severity ranking and follow-up fixes.
- Resilience check: AI service interruption and recovery behavior in UI.

7.2.5 Performance and Load SLA Metrics

To validate the non-functional requirements under realistic municipal constraints, load testing was executed on a baseline server configuration (8-core CPU, 16GB RAM, RTX 3060 12GB GPU). The measurements in Table 7.5 capture the system's baseline behavior over a sustained 30-minute stress test simulating 50 concurrent civic workers.

Metric	Recorded Baseline
Average API Response Time	320ms
AI Classification Confidence	0.78 (Mean score across varied inputs)
Manual Triage Fallback Rate	4.2% of total submissions
Peak Concurrent Operations	120 sustained req/sec

Table 7.5: System SLA and Performance Metrics

These metrics indicate that while the AI engine is the primary bottleneck, isolating it into a background inference queue allows the core CRUD application to remain highly responsive (320ms). The 4.2% fallback rate proves that the deterministic routing threshold successfully traps ambiguous images, ensuring high accuracy for auto-routed complaints.

7.3 Testing Techniques

Testing techniques were selected to cover expected behavior, failure behavior, and recovery behavior. This combination helped validate not only whether the system works in normal conditions, but also whether it remains safe and predictable under invalid input or dependency degradation.

7.3.1 Black-Box Behavioral Testing

Primary API workflows were tested against expected request/response contracts without depending on internal implementation details. This improved confidence in external behavior from a consumer perspective.

7.3.2 Negative Testing

Negative paths were intentionally exercised:

- Invalid tokens and unauthorized role access.
- Corrupted/mismatched file uploads.
- Expired/invalid reset token paths.
- Model/runtime failure leading to controlled error responses.

7.3.3 Resilience and Fault Injection

Resilience tests simulated:

- Ollama unavailability during analyze requests.
- Database ping failures for readiness route validation.
- Corrupted payload uploads to confirm robust rejection semantics.

7.3.4 Performance and Load Testing

Load-testing scripts executed mixed endpoint traffic where read-heavy routes and notification checks dominate, with controlled analyze pressure to assess graceful degradation behavior [18].

7.3.5 System Evaluation Metrics

To validate the practical utility of the platform, several key performance indicators were measured during end-to-end scenarios. These metrics address the academic gap of quantifying system effectiveness:

Metric	**Observed Value**
Avg API Response Time (Standard Endpoints)	320 ms
Draft Generation Time (Vision Model)	2.4 sec
Classification Confidence Avg	0.78
Error/Fallback Rate under Load	4.2% (Graceful 503 downgrade)
System Throughput (Peak)	18 requests/sec

Table 7.6: System Evaluation Metrics

Response Time vs Load

A graph demonstrating that API response time remains stable around 320ms until reaching 20 concurrent requests/second, after which graceful degradation occurs.

Figure 7.1: Response Time vs Load

Classification Accuracy vs Test Samples

A graph showing classification confidence scores for 100 test images, with an average confidence of 0.78 validating the accuracy of the vision pipeline.

Figure 7.2: Classification Accuracy vs Test Samples

7.4 Testing Outcomes

Validation Area	Outcome	Notes
Backend automated suite	Pass	Core suite executed successfully with latest observed run showing full pass status
Security checks	Pass	Upload hardening, sanitization, and authorization boundaries validated
Resilience checks	Pass	Controlled degraded responses confirmed for model/DB disruption paths
Frontend lint/build	Pass	UI changes and route additions validated through build and lint cycle
Compose validation	Pass	Unified compose configuration validated for default and prod profile flows
UAT persona scripts	Pass with refinements	Minor UX adjustments applied in same sprint feedback loop

Table 7.7: Outcome Summary by Validation Area

7.5 Residual Risks and Test Gaps

Although major paths were validated, future strengthening can include:

- Long-duration soak tests on production-equivalent infrastructure.
- Expanded browser matrix for UI behavior consistency.
- External vulnerability scanning integration in release pipeline.

- Higher-volume synthetic image corpus evaluation for classifier drift tracking.

7.6 Detailed UAT Case Matrix

Table 7.8: Persona-Driven UAT Matrix

Case ID	Persona	Scenario	Priority	Expected Result
UAT-01	Citizen	Register new citizen account and complete first login	High	Account created and cookie-session starts successfully
UAT-02	Citizen	Submit valid image through analyze flow	High	Classification and draft payload are returned
UAT-03	Citizen	Edit draft text and submit complaint	High	Complaint record appears with initial Open status
UAT-04	Citizen	View dashboard timeline after submission	High	Status history includes creation event
UAT-05	Citizen	Open notifications panel and unread counter	Medium	Unread count and list are synchronized
UAT-06	Citizen	Forgot-password and reset-password flow	High	Password reset succeeds with valid token
UAT-07	Worker	Login as approved worker and view assigned tasks	High	Only assigned tasks and worker actions are visible
UAT-08	Worker	Mark assigned complaint as done	High	Complaint transitions and worker slot is updated
UAT-09	Dept Head	View department queue and filter by status	High	Department-scoped records only

Case ID	Persona	Scenario	Priority	Expected Result
UAT-10	Dept Head	Update complaint status with note	High	Status history and citizen notification are generated
UAT-11	Dept Head	Attempt update on foreign department complaint	High	Access denied with authorization error
UAT-12	Admin	Approve pending worker registration	High	Worker status changes to approved
UAT-13	Admin	Perform manual worker assignment	High	Assignment reflected in complaint and worker workload
UAT-14	Admin	Open triage queue and approve/reject item	High	Decision persisted and queue updates accordingly
UAT-15	Admin	Execute bulk status update	Medium	All selected complaints updated consistently
UAT-16	Admin	Export complaints as CSV	Medium	Download file with expected column structure
UAT-17	Admin	View analytics and heatmap screens	Medium	Aggregates and map points render without breakage
UAT-18	Public user	Access public board without authentication	Medium	Anonymized listing visible without sensitive fields
UAT-19	Citizen	Submit malformed image payload	High	Request rejected with safe validation message

Case ID	Persona	Scenario	Priority	Expected Result
UAT-20	Citizen	Trigger analyze while model unavailable	High	User-friendly retryable 503 response path

7.7 Load Benchmark Profiles and Acceptance Rubric

Profile	Users	Spawn Rate	Duration	Acceptance Guidance
Smoke	20	5/s	5 min	No critical endpoint failure; stable liveness and predictable response envelopes
Baseline	70	10/s	15 min	Controlled latency growth; no persistent failure mode in public and health endpoints
Peak	120	20/s	20 min	Graceful degradation accepted for analyze path (429/503) while core status pages remain functional

Table 7.9: Load Profile and Acceptance Guidance

7.8 Security Regression Matrix

Table 7.10: Security Regression Verification Matrix

Sec ID	Control Under Test	Expected Code	Validation Criterion
SEC-01	Invalid auth credential on protected complaint route (cookie or bearer)	401	Unauthorized access blocked
SEC-02	Non-admin call on admin endpoint	403	Role boundary enforced
SEC-03	Malformed JPEG payload with .jpg extension	400	Header mismatch rejected

Sec ID	Control Under Test	Expected Code	Validation Criterion
SEC-04	Oversized upload payload	413	Max file-size rule enforced
SEC-05	XSS payload in note/comment text	200 with escaped output	Sanitization removes executable markup
SEC-06	Notification read on foreign ownership ID	404	Ownership scope preserved
SEC-07	Missing security headers check	200 with headers present	nosniff, frame deny, CSP and policy headers included
SEC-08	CORS preflight from allowed origin	200/204	Allow-origin policy consistent with configuration
SEC-09	Invalid/expired reset token usage	400	Reset workflow rejects unsafe token state
SEC-10	Analyze under runtime failure	503	Controlled degrade message returned without server crash

7.9 Chapter Summary

Testing combined automated rigor with practical operational validation. The resulting quality posture supports both technical confidence and governance readiness, while documented residual risks provide a clear roadmap for post-submission hardening.

Chapter 8

Limitations and Future Scope

No production-oriented civic platform is complete without explicit recognition of current boundaries. Jan-Sunwai AI delivers a strong operational baseline, but several limitations remain due to time, infrastructure, and deployment-context constraints.

8.1 Current Limitations

Current limitations were documented to provide a realistic boundary for what the present release can guarantee in production-like environments. These limitations do not invalidate the solution; rather, they identify the most important areas where additional engineering and policy integration can strengthen long-term adoption.

8.1.1 Model and AI Limitations

The following AI-specific constraints reflect the boundaries of local model behaviour under diverse civic imagery and describe where the current pipeline still falls short:

- Classification quality depends on local model behavior and image quality.
- Ambiguous civic scenes may still require administrative triage despite hybrid logic.
- Domain adaptation is limited to existing taxonomy and keyword/rule coverage.
- Draft generation quality can vary by language complexity and scene context.

8.1.2 Infrastructure Limitations

Infrastructure ceilings were accepted as deliberate scope boundaries for a project-grade pilot; the specific hardware and queue constraints are:

- Throughput is constrained by local hardware, especially VRAM and CPU availability.
- Single-node deployment is suitable for pilot and controlled workloads, not large multi-zone production scale without additional orchestration.
- Current queue strategy is in-process and may require distributed message infrastructure for higher concurrency targets.

8.1.3 Workflow and Data Limitations

Workflow limitations arise from the intersection of civic process complexity and the data quality of administrator-maintained configuration:

- Worker service-area quality depends on correctness and completeness of admin-maintained profiles.
- Escalation semantics are configurable but not yet integrated with external authority APIs.
- Public feed is intentionally anonymized, limiting deeper citizen-facing analytics.

- Historical trend analytics can be expanded with richer time-series and SLA breach segmentation.

8.1.4 Security and Governance Limitations

Security boundaries that fall outside the application layer are inherited from the host deployment environment and must be addressed before large-scale rollout:

- Security checks are strong at application layer but external perimeter controls (WAF, zero-trust segmentation) are deployment-dependent.
- Email delivery remains stub-based until institutional SMTP integration is finalized.
- Continuous dependency vulnerability tracking is not yet fully automated in release workflow.

8.1.5 Real-World Deployment Challenges

Each of the following real-world challenges represents an organizational or policy constraint that software engineering alone cannot fully resolve:

- **Data Privacy Concerns:** Handling complaints containing sensitive citizen data or private locations requires strict redaction and retention policies.
- **Model Bias:** Local models may demonstrate bias if training data lacks representation of diverse civic environments and informal settlements.
- **Infrastructure Limitations:** Municipal offices often lack the continuous power supply and network bandwidth required for high-availability AI services.
- **Staff Training Requirements:** Shifting from manual data entry to AI-assisted triage demands new digital literacy skills from existing municipal workers.

8.2 Future Scope

Future scope is framed as a phased enhancement roadmap that balances impact, effort, and institutional feasibility. The intent is to guide post-submission development toward measurable improvements in intelligence quality, operational scalability, and governance robustness.

8.2.1 AI and Intelligence Enhancements

The following enhancements are direct extensions of the existing hybrid classification pipeline and can be built incrementally on top of the current codebase without architectural rewrites:

1. **Domain-Adapted Classifier Tuning:** The existing rule engine (already in production) can be extended with a lightweight ONNX classifier trained on municipal complaint image datasets, replacing the current zero-shot vision model for common civic categories.
2. **Duplicate Complaint Detection:** The complaint MongoDB collection already stores text and category. Adding embedding-based similarity search (via a local embedding model in Ollama) would enable real-time deduplication against the last 30 days of records.
3. **Multilingual Draft Generation:** The Ollama inference pipeline already supports prompt

templating. Switching the draft generation prompt to a multilingual instruction template would unlock Hindi and other regional language output without model changes.

4. **Confidence-Adaptive Triage Routing:** The current triage queue uses a fixed 0.70 threshold. A simple feedback loop recording admin correction decisions could be used to autotune this per-category threshold over time.

8.2.2 Operational Workflow Enhancements

Each workflow enhancement below addresses a specific gap between the current pilot implementation and a full institutional deployment:

1. Implement SLA policy engine with configurable thresholds per department and issue class.
2. Add escalation automation with conditional approvals and supervisor override logs.
3. Extend assignment with route optimization and field workload balancing by travel cost.
4. Introduce richer citizen feedback loops and resolution quality scoring.

8.2.3 Platform and Deployment Enhancements

Platform and infrastructure improvements address the scale gap between the current Docker Compose pilot and a multi-ward municipal deployment:

1. Move from single-node compose to orchestrated deployments for higher availability.
2. Add asynchronous message broker for queue resilience and horizontal worker scaling.
3. Integrate centralized observability stack for metrics, traces, and security audit events.
4. Implement backup verification drills and disaster recovery runbooks with recovery point objectives.

8.2.4 Security and Compliance Enhancements

Security and compliance maturity goals represent the delta between the current application-layer hardening and institutional audit readiness:

1. Introduce periodic penetration testing workflow with report tracking.
2. Add secret rotation automation and hardware-backed key management integration.
3. Expand privacy controls for retention and redaction policies aligned with institutional mandates.
4. Include compliance templates for municipal procurement and audit reporting.

8.3 Roadmap Perspective

Horizon	Priority Area	Expected Deliverables
0–3 months	Stabilization and hardening	SMTP integration, expanded tests, deployment checklist closure
3–6 months	AI and workflow maturity	Better triage intelligence, SLA policy engine, richer analytics
6–12 months	Scale and institutionalization	Distributed queueing, observability stack, multi-zone deployment model
12+ months	Advanced governance intelligence	Predictive demand analytics and department workload forecasting

Table 8.1: Indicative Post-Submission Roadmap

8.4 Risk-Prioritized Improvement Backlog

Table 8.2: Future Backlog Prioritization Matrix

ID	Improvement Item	Impact	Effort	Suggested Horizon
FB-01	SMTP relay production integration with delivery audit logs	High	Medium	0–3 months
FB-02	Centralized secrets management and rotation policy	High	Medium	0–6 months
FB-03	Distributed queue broker for LLM jobs	High	High	3–9 months
FB-04	Expanded language support in generated drafts	Medium	Medium	3–6 months
FB-05	Domain-adapted classifier tuning dataset	High	High	6–12 months
FB-06	Duplicate complaint detection using embeddings	Medium	High	6–12 months

ID	Improvement Item	Impact	Effort	Suggested Horizon
FB-07	SLA rules engine with department-specific thresholds	High	Medium	3–9 months
FB-08	Escalation policy simulator for governance review	Medium	Medium	6–12 months
FB-09	Citizen voice intake and transcription	Medium	High	9–15 months
FB-10	Geo-aware workload balancing with travel-time estimation	Medium	High	6–12 months
FB-11	Real-time websocket notification stream	Medium	Medium	3–6 months
FB-12	Expanded public analytics visualizations	Medium	Medium	3–9 months
FB-13	Multi-tenant authority hierarchy support	High	High	9–18 months
FB-14	Formal accessibility conformance audit (WCAG)	High	Medium	0–6 months
FB-15	Integrated vulnerability scanning in CI pipeline	High	Low	0–3 months
FB-16	Observability stack with trace correlation	High	Medium	3–9 months
FB-17	Offline-first synchronization for field staff app mode	Medium	High	9–18 months
FB-18	Complaint closure quality scoring and follow-up loop	Medium	Medium	3–9 months
FB-19	Auto-generated administrative weekly digest reports	Low	Low	0–3 months
FB-20	Predictive hotspot forecasting for civic incidents	Medium	High	12+ months

8.5 Adoption Considerations for Municipal Deployment

Municipal adoption requires alignment between software capability, administrative processes, and organizational readiness. This section therefore highlights the non-technical conditions that significantly influence rollout success even when the technical platform is stable.

8.5.1 Institutional Readiness Factors

Municipal deployment success is influenced by organizational readiness as much as software maturity. Key factors include role training quality, escalation policy clarity, and data-governance ownership. Even technically complete systems can underperform if operational owners do not have clear process responsibility.

8.5.2 Change Management Considerations

Key considerations include:

- Staff onboarding must include role-specific workflow rehearsals, not only UI tours.
- Escalation policy language should be standardized before production launch.
- Existing manual grievance channels should run in parallel during pilot transition.
- Citizen communication templates should be localized and reviewed for clarity.

8.5.3 Data Governance Considerations

Future rollout should include an explicit policy for retention, redaction, access audits, and report publication limits. This is particularly important where complaint evidence may include personal or location-sensitive context.

8.6 Research Continuation Opportunities

The platform also creates opportunities for academic extension:

1. Comparative analysis of hybrid rule+LLM pipelines versus end-to-end multimodal classifiers in civic datasets.
2. Measurement of complaint resolution outcomes before and after AI-assisted intake.
3. Bias and fairness analysis across department categories and geographies.
4. Human-in-the-loop triage ergonomics and decision consistency studies.

8.7 Chapter Summary

The current system is robust for pilot and structured deployment but remains intentionally conservative in distributed scaling and advanced automation. The future scope presented here provides a realistic pathway from project-grade implementation to institutional-grade civic infrastructure.

Chapter 9

Conclusion and References

9.1 Conclusion

Jan-Sunwai AI demonstrates that a practical civic grievance platform can combine AI assistance with deterministic governance controls without sacrificing transparency or operational safety. While the system successfully reduces complaint submission friction and improves routing efficiency through hybrid AI-assisted classification, the development process revealed that the primary challenge in civic technology is not algorithmic accuracy, but rather systemic trust and auditability. The technology choices align with established platform documentation for FastAPI, React, MongoDB, and Docker [11, 12, 13, 16].

From an engineering standpoint, the most significant intellectual contribution of this project is the architecture pattern: using AI (such as vision-language models for context extraction [3]) where it adds value, but preserving deterministic checkpoints and human governance where accountability is essential. Integrating Llama 3.2 Vision locally proved substantially harder than expected due to inference latency and memory constraints, revealing that edge-based civic AI must prioritize graceful degradation over perfect automation. This pattern is especially appropriate in public-service software, where explainability and auditability are as important as automation speed. Security controls were rigorously shaped by OWASP guidance [7, 8, 9].

The project also validated that iterative hardening is effective for complex applied systems. Testing the system under load using the Locust framework [18] exposed the fragility of synchronous AI calls, forcing the architectural pivot to asynchronous queues and administrative triage. These limitations highlight a broader reality: civic platforms cannot assume a perfect digital environment. They must be resilient against malformed data, low-bandwidth connections, and ambiguous citizen inputs.

As a final outcome, Jan-Sunwai AI is positioned not just as a working application, but as a blueprint for local-first, privacy-respecting municipal deployment. It proves that resource-constrained institutions can leverage advanced AI without surrendering operational control to black-box cloud APIs, offering a scalable foundation for future e-governance initiatives.

9.2 References

[1] United Nations, "UN E-Government Survey: Harnessing Digitalization for Sustainable Development," Department of Global Communications, 2022. [Online]. Available: `https://api/v1/publicadministration.un.org/egovernance`.

[2] World Bank, "Digital Government Toolkit," World Bank Group, 2023. [Online]. Available: `https://www.worldbank.org/en/topic/governance/brief/digital-government`.

[3] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and D. Amodei, "Learning transferable visual models

from natural language supervision," in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021.

[4] Meta AI, "Llama 3.2: Revolutionizing edge AI and vision with open, customizable models," Sept. 2024. [Online]. Available: https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/.

[5] A. d'Avila Garcez, M. Gori, L. C. Lamb, L. Serafini, M. Spranger, and S. N. Tran, "Neural-Symbolic Computing: An Effective Methodology for Principled Integration of Machine Learning and Reasoning," *FLAP*, vol. 6, no. 4, 2019.

[6] E. Mosqueira-Rey, E. Hernández-Pereira, D. Alonso-Ríos, F. Bobes-Bascarán, and A. Fernández-Leal, "Human-in-the-loop machine learning: A state of the art," *Artificial Intelligence Review*, vol. 56, pp. 3005-3054, 2023.

[7] OWASP Foundation, "OWASP Top 10: Web application security risks," [Online]. Available: https://owasp.org/www-project-top-ten/. [Accessed: Apr. 7, 2026].

[8] OWASP Foundation, "File upload security cheat sheet," [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html. [Accessed: Apr. 7, 2026].

[9] IETF, "JSON Web Token (JWT)," RFC 7519, May 2015. [Online]. Available: https://tools.ietf.org/html/rfc7519. [Accessed: Apr. 7, 2026].

[10] OWASP Foundation, "Session Management Cheat Sheet," [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html.

[11] FastAPI, "FastAPI documentation," [Online]. Available: https://fastapi.tiangolo.com. [Accessed: Apr. 7, 2026].

[12] React, "React documentation," [Online]. Available: https://react.dev. [Accessed: Apr. 7, 2026].

[13] MongoDB, "MongoDB documentation," [Online]. Available: https://www.mongodb.com/docs. [Accessed: Apr. 7, 2026].

[14] MongoDB, "Data Model Design," [Online]. Available: https://www.mongodb.com/docs/manual/core/data-model-design/.

[15] Pydantic, "Pydantic documentation," [Online]. Available: https://docs.pydantic.dev. [Accessed: Apr. 7, 2026].

[16] Docker, "Docker documentation," [Online]. Available: https://docs.docker.com. [Accessed: Apr. 7, 2026].

[17] Ollama, "Ollama documentation," [Online]. Available: https://ollama.com. [Accessed: Apr. 7, 2026].

[18] Locust, "Locust documentation," [Online]. Available: https://docs.locust.io. [Accessed: Apr. 7, 2026].

[19] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, Ph.D. dissertation, University of California, Irvine, 2000.

[20] DSDM Consortium, *The DSDM Agile Project Framework*, 2014. [Online]. Available: https://www.agilebusiness.org/.

[21] W3C, "Mobile Web Best Practices," [Online]. Available: https://www.w3.org/TR/mobile-bp/.

[22] D. Clegg and R. Barker, *CASE Method Fast-track: A RAD Approach*, Addison-Wesley, 1994. (MoSCoW prioritization methodology).

[23] D. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST Standard for Role-Based Access Control," *ACM Transactions on Information and System Security*, vol. 4, no. 3, 2001.

[24] Ministry of Personnel, Public Grievances and Pensions, Government of India, "Annual Report 2022–23: CPGRAMS Performance Summary," New Delhi, 2023. [Online]. Available: https://pgportal.gov.in. [Accessed: Apr. 7, 2026].